

OCELOT: Inference-Leakage Budgets for Privacy-Preserving LLM Agents

Jin Xie, Songze Li

Abstract—Large language model (LLM) agents increasingly act on a user’s behalf—reading personal files, calling tools, transacting with external services—possibly leaking personally identifiable information (PII) across trust boundaries at every step. Privacy here is a property not of a single output but of an entire trajectory, and three properties make it hard: leakage is cumulative, as individually innocuous releases accumulate across honest-but-curious or colluding sinks into inferences about a protected secret; bidirectional, as a malicious observation can inject instructions that turn the agent’s own reasoning model against the user; and task-dependent, as the same field is necessary for one recipient yet gratuitous for another. Per-release contextual-integrity filters, information-flow controls, and posterior-leakage monitors each address part of this but none controls cumulative, inference-based leakage at runtime. We recast agent privacy as *posterior-risk control* and present OCELOT, a runtime mediator that budgets how much an adversary’s belief about a secret may improve across a trajectory, rather than filtering outputs. Its mechanism, *Witness-Verified Declassification*, separates judgment from trust: an untrusted, locally fine-tuned defender model inspects each candidate release and emits structured evidence—labeled atoms and proposed declassification operators—which a deterministic verifier audits, charging a certified min-entropy cost for the chosen variant and authorizing the least-disclosing useful release under a sink-trust-weighted budget recorded on a tamper-evident ledger. Because every unverified atom is costed at worst-case risk, a jailbroken defender yields conservative fallback, never silent leakage. This gives a two-layer guarantee: budget non-exceedance holds deductively, independent of any LLM, while semantic privacy holds empirically, to the degree calibrated costs upper-bound true adversarial inference. Across diverse agent benchmarks and recent defenses, OCELOT attains significantly lower leakage at higher task utility, resists adaptive injection, jailbreak, cumulative inference, and sink collusion, and adds only modest overhead.

1. Introduction

An LLM agent that books a flight does not merely answer a question; it *acts*. To finish the task it reads the user’s saved profile, queries a travel broker for fares, submits a passenger record to an airline, and authorizes a charge with a payment processor—disclosing a different slice of the user’s identity to a differently-trusted party at each step. This action-taking posture, standard in production agent harnesses (Claude Code, OpenAI Codex, Cursor) [1], [2], [3], [4], is what makes agents useful, and a new privacy

liability: an agent is a conduit through which personally identifiable information (PII) flows continuously, across trust boundaries, on the user’s behalf.

Prior work on language-model privacy centers on what a *model* memorizes or what a *single* output reveals [5], [6]. Agent privacy is a different object: exposure is a property of an entire trajectory of releases to many sinks, and what matters is not whether any message contains an identifier but how much an adversary observing the trajectory can *infer* about a protected secret. A defense that inspects releases in isolation cannot see this—recent multi-agent benchmarks confirm the dominant residual leakage is accumulated and indirect, not any single offending message [7].

Three properties, traced in Figure 1, make agent privacy mediation hard. **(P1) Leakage is cumulative:** individually innocuous releases combine—at one honest-but-curious sink, or across colluding sinks—to reconstruct a secret no release named. **(P2) Leakage is bidirectional:** information escapes when the agent ingests observations, not only when it acts; a tool can return content that injects instructions, subverting the task-aligned but untrusted reasoning model into leaking more on later turns [8]. **(P3) Leakage is task-dependent:** the same field can be required by one sink and gratuitous for another in the same trajectory, so no universal safe answer exists. Together these defeat the per-release, binary, and monitoring defenses deployed today.

The question a mediator must answer at runtime is therefore not “allow or deny?” but: *given the task, a candidate release, the destination sink, the trajectory so far, and the remaining privacy budget, what is the least-disclosing useful variant that can be authorized with auditable evidence?* This demands three things no prior defense provides together: a quantitative, composable notion of leakage that accumulates across a trajectory; a mechanism that chooses among semantic-granularity variants rather than a binary verdict; and an authorization decision whose soundness does not depend on the correctness of any LLM. Existing runtime defenses each supply one of these and miss the rest (§2).

We present **OCELOT**¹, a runtime mediator that recasts agent privacy as *posterior-risk control*: it budgets how much an adversary’s posterior over a secret may improve across a trajectory, rather than filtering outputs. Its mechanism, *Witness-Verified Declassification* (WVD), resolves the central tension—semantic judgment needs an LLM, yet no LLM can be trusted in an adversarial loop—by separating the two. An untrusted, locally fine-tuned *de-*

1. **OCELOT:** *Operator-Calibrated, Evidence-verified, Leakage-budgeted Online Trajectory mediator.*

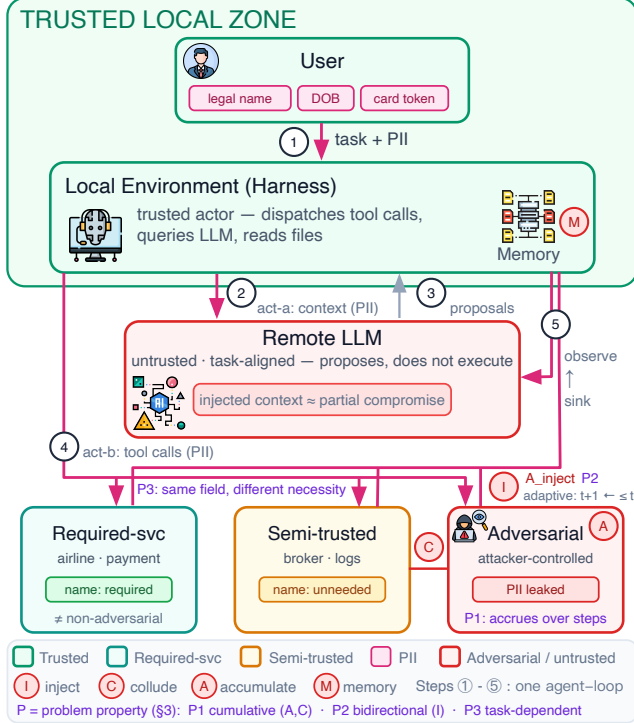


Figure 1. Privacy exposure in one LLM-agent loop: a user’s PII (pink) crosses trust boundaries to the remote LLM (act-a) and to external sinks (act-b), with observations returning inward (observe). Three properties (§3) defeat per-release defenses—cumulative accrual and collusion (P1, glyphs A/C), bidirectional injection (P2, I), and task-dependent necessity (P3); unverifiable sinks default to ADVERSARIAL.

fender applies a versioned rubric to each candidate release and emits structured *evidence*: labeled atoms and a proposed sequence of declassification operators. A deterministic, trusted *verifier* audits this evidence, computes a certified min-entropy cost for the proposed variant from calibrated per-operator bounds, and authorizes the least-disclosing useful variant under a sink-trust-weighted budget recorded in a Merkle-chained ledger. The defender is *honest-but-jailbreakable*: because the verifier costs any unlabeled or mislabeled atom at top risk, a coerced defender yields *fallback*, never silent leakage. The result is a two-layer guarantee we state honestly—budget non-exceedance holds *deductively* by ledger arithmetic, independent of any LLM, while the semantic-privacy implication holds *empirically*, to the extent calibrated costs upper-bound true adversarial inference, which we measure rather than assume.

Contributions.

- *Problem and goal.* We formalize privacy mediation for LLM agents around properties P1–P3 and a two-layer security goal separating a deductive budget guarantee from an empirical, calibration-relative privacy claim (§3).
- *Mechanism.* We design WVD: an untrusted defender producing verifiable evidence under grammar-constrained decoding, and a deterministic verifier that authorizes least-disclosure variants via a four-phase procedure with conservative defaults,

structural-field obligations, and a Merkle-chained leakage ledger (§4).

- *Costs and training.* We calibrate per-operator min-entropy costs as adversarial-inference upper-confidence bounds, audit them with a bounded-treewidth credal circuit, and train the defender with a GRPO reward computed by the verifier itself, so the policy optimizes against the exact deployment-time check (§4.3, §4.2).
- *Benchmark and protocols.* We release OCELOT-Traj-500 and three pre-registered protocols—calibration, escape-rate, and rubric construction—for reproducible evaluation (Appendices D.1–D.1).
- *Evaluation.* Across five benchmarks and five recent defenses, OCELOT attains the lowest leakage *and* highest task success on every benchmark, records zero budget exceedances over 22,585 trajectory-instances, and degrades far more gracefully than every non-denial baseline under injection, jailbreak, cumulative inference, and sink collusion—at 18% wall-clock overhead (§5).

2. Related Work

Production LLM agents run a harness that brokers all I/O under permissions [2], [3], and their dominant risk is a task-aligned model executing instructions injected into external content; AgentDojo [8] systematizes such attacks and serves as our injection benchmark, and the structural defense it motivates—stripping authority from untrusted data rather than detecting it—is one we reuse on the observe channel. The privacy threat is subtler. Beyond verbatim memorization [5], LLMs can *infer* sensitive attributes from benign text [6], so the quantity that matters is not whether a message contains an identifier but how much an adversary can infer across a trajectory. ConfAide [9] casts contextual integrity [10] as tiered single-release judgments, whereas AgentLeak-2026 [7] shows that in multi-agent trajectories the dominant residual leakage is cumulative and indirect; we evaluate on both, using single-release ConfAide to stress-test rather than flatter our trajectory framing.

Runtime defenses fall into three families, and our baselines span all three. Per-release contextual-integrity filters—FlexGuard [11] and PrivacyChecker [12]—judge whether each disclosure is appropriate but keep no cross-trajectory state, so independently “appropriate” releases can jointly reconstruct a secret (P1), and their binary verdicts cannot trade granularity for utility (P3). Information-flow and capability systems—CaMeL [13] and RTBAS [14]—gate flows across trust boundaries with strong worst-case safety but answer only *whether* information may flow, not *how much*: a field is released at full fidelity or denied, the characteristic IFC trade of low leakage bought with heavy over-denial, which makes CaMeL our denial-based reference on the privacy axis. Closest to our cost model, AGENTDAM [15] estimates how much a release improves an adversary’s posterior but is a *monitor*—it quantifies risk without enforcing a budget and forwards injected observations unmediated; we compare

against both its monitoring configuration and a steelmanned threshold-and-block variant (Appendix D.2), yet thresholding a per-release estimate still cannot bound a cumulative sum. Training-time regularization [16] bounds leakage during fine-tuning and is complementary but orthogonal, requiring weight access and offering no runtime control over which field reaches which sink at what granularity.

Our mechanism builds on established theory: per-operator costs are min-entropy leakage and Bayes-vulnerability bounds from quantitative information flow [17], [18], our trajectory accounting parallels a privacy odometer in differential-privacy composition [19], the independent audit is a credal sum-product circuit [20], [21], and rubric obligations are discharged by an SMT solver [22], [23]. The contribution is their synthesis into a runtime mediator whose budget guarantee is independent of LLM behavior. Each prior family supplies one ingredient; to our knowledge OCELOT is the first runtime agent privacy defense to provide all three at once—*controlling* rather than merely measuring cumulative inference risk, acting at *semantic granularity* rather than allow/deny, and grounding authorization in a *deterministic verifier* whose guarantee does not depend on any LLM.

3. Problem Formulation

3.1. The Privacy-Mediation Problem

An LLM agent executes a user-issued task as a multi-step trajectory: it reads observations from tools and the environment, maintains memory across turns, and emits actions to external sinks — APIs, logs, third-party services, and other agents. Each step may cross a trust boundary, so a privacy defense must answer, at runtime, what may be released to whom, in what form, and at what cost. We call this the *privacy-mediation problem for LLM agents* and identify three properties distinguishing it from problems prior defenses address.

(P1) Leakage is cumulative. Individual releases may carry no explicit identifier, yet several weak releases let a sink (or colluding set of sinks) infer a protected secret. Per-release filters cannot detect this, and recent benchmarks confirm that the dominant residual leakage in multi-agent systems flows through accumulated, indirect disclosures [7].

(P2) Leakage is bidirectional. Information escapes not only when the agent acts toward a sink but also when it ingests observations: a tool may return content crafted to extract disclosure on later turns, or to inject instructions that cause subsequent releases to leak more [8].

(P3) Leakage is task-dependent. The same field can be *necessary* for one sink and *unnecessary* for another in the same trajectory: a passenger’s date of birth is needed by an airline at ticket issuance but not by a flight-search broker. No universal safe answer exists; the question is what disclosures are necessary for *this* task, at *what* granularity, to *which* sink.

Prior defenses miss (P1)–(P3). IFC- and capability-style agent defenses [13], [14] decide whether information may

flow, not how much or at what granularity. Contextual-integrity checkers [11], [12] judge appropriateness one release at a time, without cumulative accounting. Posterior-leakage estimators [15] measure leakage but do not control it. Information-theoretic training-time regularization [16] bounds leakage via fine-tuning, requiring weight access and offering no runtime mediation. None jointly answer:

Given the current task, candidate release, sink, trajectory history, and remaining privacy budget, what is the least-disclosing useful variant of the release that can be authorized with auditable evidence?

Answering it requires a quantitative composable notion of leakage that accumulates across a trajectory, a choice among semantic-granularity variants rather than binary allow/deny, and an authorization whose soundness does not depend on any LLM.

3.2. System Setting and Information Flow

We consider an LLM-agent system following the harness architecture of production agents (Claude Code, OpenAI Codex, Cursor, OpenHarness) [2], [3]. Four parties are relevant; the mediator this paper introduces is a fifth component inside one of them.

User. Issues a task and trusts the local environment. Holds a typed secret S with a per-task prior $P(S)$, default uniform.

Local environment (harness). Trusted software on the user’s machine. It brokers all communication — reading local files, managing persistent memory, querying the remote LLM, dispatching tool calls, handling permissions and sandboxing — and is the *actor* in the agent loop: the LLM only proposes; the harness executes.

Remote LLM. The reasoning component, reached over an API (e.g., `/v1/messages`, `/v1/responses`) [3]. It receives task context, prior observations, and tool schemas and returns text, tool-call proposals, and reasoning; it does not execute. It is task-aligned but *untrusted*: content injected into its context, typically through observations returned by sinks, can subvert its behavior [3], [8].

External sinks. Tools, APIs, websites, MCP servers, logs, memory stores, and other agents the harness contacts on the user’s behalf. Each carries a declared trust class: ADVERSARIAL (under attacker control); SEMI-TRUSTED (honest-but-curious, may log but not actively attack); and REQUIRED-SERVICE (an authenticated endpoint, e.g., a payment processor, whose disclosure is required for the task). The classes are policy weights, not safety assumptions: REQUIRED-SERVICE does *not* mean non-adversarial, and unverifiable sinks default to ADVERSARIAL.

Local mediator. The component this paper introduces, inside the harness: a local fine-tuned LLM (the defender, §4.2) and a deterministic verifier (§4.4), intercepting every information flow crossing the harness boundary toward the remote LLM or external sinks.

Trust-asymmetric channels. Information crossing the harness boundary divides into four channels, distinguished by the trust relation between source and destination rather than by topology:

- **Act channel.** Trusted local content flowing outward, in two legs: *harness* $\rightarrow$ *remote LLM* (task context, file contents, prior observations sent to the API) and *harness* $\rightarrow$ *external sink* (tool calls).
- **Observe channel.** Content from external sinks flowing inward through the harness to the remote LLM.
- **Memory channel.** Trajectory state written to a persistent store, readable by future trajectories with possibly different trust.
- **User channel.** Confirmations and disclosures back to the user; audited but unbudgeted.

The four channels aggregate to the seven topological channels of [7], which we use only for benchmark comparison.

3.3. Adversary Model

Capability. The adversary controls every ADVERSARIAL-class sink and may inject content into any observation reaching the remote LLM through the harness. It observes every act-channel release to ADVERSARIAL and SEMI-TRUSTED sinks and every memory-channel write later readable by such sinks, and may collude across them. It is *adaptive*: injected content on turn $t+1$ may depend on releases observed up to turn t . No compute bound is assumed; inference may combine released variants with arbitrary public background knowledge. We do not assume the provider’s infrastructure is compromised, but a remote LLM whose context contains injected instructions behaves indistinguishably from a partly-compromised one, and the defense must account for this.

Goal. The adversary aims to reduce the min-entropy of S given its view below a target threshold; equivalently, to drive $\Pr[S=s \mid \text{view}]$ above a threshold for some s .

Component trust. The harness and the mediator inside it are trusted at the level of standard system integrity (no kernel compromise, no key exfiltration); orthogonal hardening is out of scope. The remote LLM is *untrusted but task-aligned*: its proposed tool calls are inputs to mediation, not authorizations. The defender is also an LLM and is modeled as *honest-but-jailbreakable*: its weights are trusted (operator-controlled fine-tuning), but its outputs are not, since injection in its input may with bounded probability coerce misleading content. The mediator therefore treats the defender as a constrained witness producer under deterministic verification, not a classifier whose verdict is dispositive.

Out of scope. A compromised user; compromise of the harness’s runtime integrity (orthogonal TEE/signed-binary infrastructure); compromise of the LLM provider’s infrastructure or model weights; physical-layer side channels; denial-of-service against the mediation pipeline; and inference attacks exploiting *only* public background knowledge without combining it with any released variant. The last

exclusion is consistent with the unbounded priors granted above: a charged leakage is one in which some released variant moves the posterior.

3.4. Security Goal

We formulate the security goal in two regimes — deductive (Layer 1a) and empirical (Layers 1b and 2) — matching what a defense can honestly guarantee.

Layer 1a (deductive): budget non-exceedance. For every trajectory of length k it authorizes and every adversary in the class above, the system must enforce

$$\sum_{i=1}^k \text{cost}_i \leq B, \quad (1)$$

where cost_i is the certified release cost charged at step i (defined in §4.4) and B is a fixed per-task base budget. Sink trust enters through cost_i via the trust multiplier $\mu(z)$ (§4.4), not through B . This holds without exception, by ledger arithmetic (Theorem SP-1).

Layer 1b (model-relative): cost soundness. For every authorized release with variant v , the *model-relative* leakage $\hat{L}(v; S)$ — the min-entropy reduction computed under the calibration model of Pre-Reg. A — satisfies $\hat{L}(v; S) \leq \text{cost}$, except with empirically bounded probability δ : the residual probability that the mediator’s LLM-based components emit labels passing the verifier’s validate phase while misclassifying an atom’s field outside the verifier-considered field set (measured in Pre-Reg. B, §C.2).

Layer 2 (model-relative): semantic privacy. *If, on the released variants, $\hat{L}$ upper-bounds the true adversary’s realized MAP advantage, then for every authorized trajectory*

$$H_\infty(S) - H_\infty(S \mid \text{view}) \leq B, \quad (2)$$

where *view* is the adversary’s realized observation and $H_\infty(S \mid \text{view}) = -\log_2 \max_s \Pr[S=s \mid \text{view}]$ is the realized (not channel-averaged) conditional min-entropy; the left-hand side is then the realized MAP advantage accumulated over the trajectory — the quantity the per-release costs bound.

The split is deliberate: (1) holds by ledger arithmetic regardless of how $\hat{L}$ relates to the true posterior, whereas the Layer-2 implication is an empirical claim, since no defense can deductively bound leakage against an unbounded-compute adversary with arbitrary background knowledge. A credible defense must therefore confine the LLM-dependent assumptions to the calibrated terms. The remainder of this paper presents such a defense.

4. Witness-Verified Declassification

This section presents **OCELOT**, a runtime privacy mediator realizing the security goal of §3.4 under the adversary model of §3.3. Its mechanism, *Witness-Verified Declassification* (WVD), pairs an untrusted local LLM that

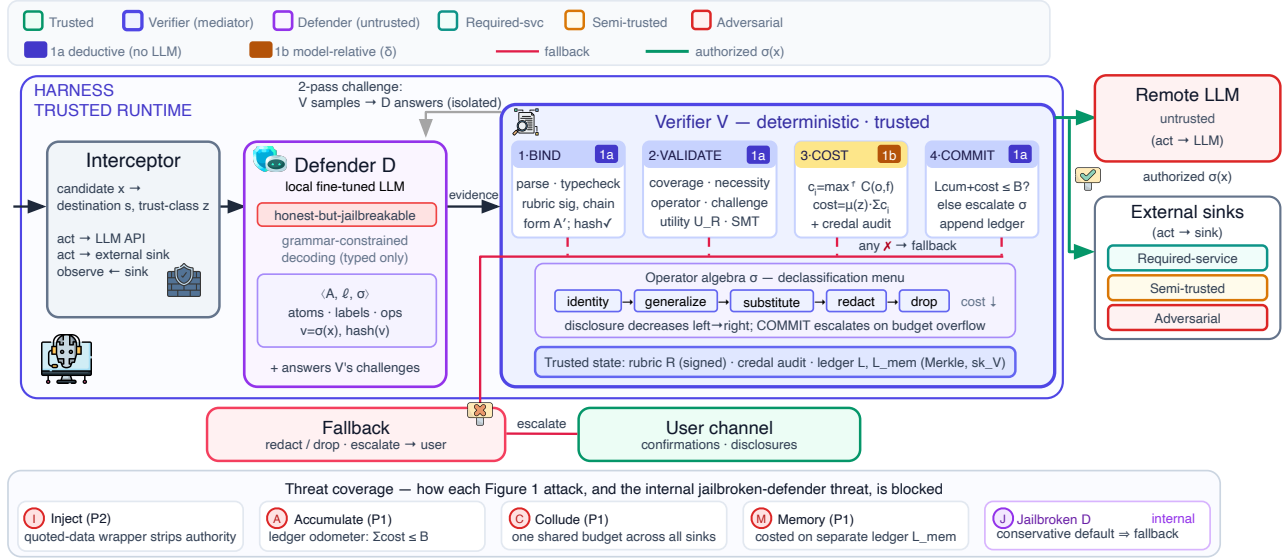


Figure 2. OCELOT architecture. For each candidate release, the untrusted defender D emits evidence $\langle A, \ell, \sigma \rangle$; the deterministic verifier V audits it in four phases against its trusted state and authorizes the least-disclosing variant $\sigma(x)$ or falls back. Bind, Validate, and Commit are deductive (no model dependence); Cost is model-relative (residual δ). The bottom band maps each Figure 1 threat (I,A,C,M) and the internal jailbroken-defender threat (J) to its defense; any failed check yields fallback, never silent release.

applies a versioned rubric to each candidate release — emitting structured atom labels and an operator proposal — with a deterministic verifier that checks the labels against rubric obligations, computes a certified leakage cost, and authorizes the least-disclosing useful variant under a per-trajectory, sink-trust-weighted budget. Figure 2 shows the architecture; Algorithm 1 the authorization procedure for one mediated release.

4.1. Architecture and Rubrics

The defender D (§4.2) and verifier V run inside the harness; V holds a versioned rubric library $\mathcal{R}$, a per-trajectory Merkle-chained ledger $\mathcal{L}$, a credal posterior audit engine, and a signing key sk_V .

What OCELOT mediates. The three boundary-crossing flows of §3.2 (act legs a and b, and the observe channel) are mediated by one procedure: whenever the harness is about to send a message across the boundary in either direction, V intercepts it before transmission. We write x for the candidate release, d for the destination (an external sink or the remote LLM endpoint), z for its declared trust class, and S for the user’s secret. Algorithm 1 summarizes the procedure; its symbols are defined as they arise below.

Division of labor. Open-vocabulary semantic classification — “does this span imply a sensitive attribute?” — is a task LLMs do well and deterministic code does not, so the defender atomizes and labels while the verifier audits: checking the atom set against rubric coverage obligations and structural-field obligations from trusted tool schemas, verifying label-operator consistency, and computing certified costs. Omission attacks — a malicious defender silently dropping a sensitive atom — are countered by four mechanisms in concert: rubric coverage obligations (below),

Algorithm 1 WVD authorization for one candidate release. Lines 1–3 resolve the rubric set and obtain the defender’s evidence (two isolated passes); lines 4–6 are BIND, line 7 VALIDATE, lines 8–9 COST (including the credal audit), and lines 10–11 COMMIT. Named procedures are defined in surrounding prose.

Require: candidate x , destination d with trust class z , ledger $\mathcal{L}$

- 1: $R \leftarrow \text{RESOLVERUBRICSET}(x, d)$ $\triangleright$ set-valued; conservative join
- 2: $(A, \ell, \sigma) \leftarrow D.\text{APPLYRUBRIC}(x, R)$
- 3: $c^{\text{ch}} \leftarrow D.\text{ANSCHALLENGES}(V.\text{SAMPLECHALLENGES}(R), x)$
 $\triangleright$ second isolated pass; clauses chosen by V
- 4: $A' \leftarrow A \cup \text{STRUCTURALFIELDDATOMS}(x, R)$
- 5: $\ell' \leftarrow \text{APPLYCONSERVATIVEDFAULT}(\ell, A')$
- 6: **if** $\neg \text{BIND}(x, \sigma, R, \mathcal{L})$ **then return** FALLBACK
- 7: **end if**
- 8: **if** $\neg \text{VALIDATE}(\ell', c^{\text{ch}}, \sigma, A', R, U_R)$ **then return** FALLBACK
- 9: **end if**
- 10: $\text{cost} \leftarrow \text{COST}(\sigma, A', \ell', R, \mu(z))$
- 11: **if** $\text{AUDIT}(\sigma(x)) > \text{cost} + \varepsilon(R)$ **then return** FALLBACK
- 12: **end if**
- 13: **if** $L_{\text{cum}} + \text{cost} > B$ **then return** $\text{ESCALATEANDRETRY}(\sigma, R)$
- 14: **end if**
- 15: $\mathcal{L}.\text{APPEND}(e_i)$; **return** $\sigma(x)$

structural-field obligations from trusted tool schemas, adversarial omission-curriculum training (§4.2), and two-pass consistency challenges (§4.2). None is individually airtight; their combination makes the architecture defensible.

Rubric resolution is set-valued. A structured task typer in V ’s trusted code extracts a tuple (action_category, {field_types}, {sink_trust_class}) from *trusted_structured_metadata only*: tool schemas, endpoint identity, signed registry labels. The LLM API endpoint is itself a destination with its own declared trust class;

free-form user text contributes candidate tags but cannot select a less restrictive rubric on its own. The tuple keys a lookup returning the set R of rubrics matching the structural signature. Under ambiguity (multiple matches), V enforces the intersection of admissible operators, the maximum of per-(operator, field) costs, and the minimum of base budgets. Under no match, a fallback rubric $r_{\perp}$ applies, admitting only REDACT and DROP with a small budget. Adversarial task phrasing therefore cannot increase disclosure.

What a rubric contains. A rubric $r \in \mathcal{R}$ for one task category is a typed, signed record with seven components: (1) the *task signature* (action category, field types, destination trust class) keying lookup; (2) the *secret-field schema* drawing from a global field ontology (e.g., LEGAL-NAME, DOB, PAYMENT-TOKEN); (3) *coverage obligations* — per-field trigger predicates (regex over structured schemas, named-entity-type constraints, lexicon matches) the verifier evaluates over the candidate, declaring that any candidate span matching a predicate must correspond to an atom with a non-NONE field label; (4) *necessity rules* — per-(field, destination-class) clauses declaring admissible necessity levels; (5) the *operator algebra* of Table 1 with per-(operator, field) calibrated costs $C(\text{op}, f)$, the generalization level k for GENERALIZE, and the template-ID list for SUBSTITUTE; (6) *budget and trust weights* — per-trajectory base budget B_0 , trust-class multiplier $\mu(z)$ ($\mu(z) \geq 1$, so the budget never under-charges leakage), and calibration margin $\varepsilon(R)$; (7) the *utility predicate* U_R over the proposed variant, evaluated deterministically by V on required-field atoms. Coverage obligations do not constitute semantic completeness; they specify the minimal flagging discipline V enforces.

How rubrics are built. Rubric authoring is a pre-registered three-stage offline process (Pre-Reg. C): domain-expert *drafting* against a template (optionally using a frontier model as a assistant rather than an authority); *validation* against a held-out corpus of ≥ 100 realistic agent trajectories per task category, scoring coverage-obligation hit rate, necessity-rule annotator agreement, and operator-algebra completeness; and *adversarial probing*, a red-team pass that attempts to construct releases satisfying all coverage obligations yet leaking, with failures triggering rubric patches and re-validation. The result is not semantic completeness but a transparent, versioned, externally auditable artifact.

Running example. A user instructs the harness: “Find a flight from San Francisco to Boston tomorrow morning under \$400 and buy it with my saved card.” Legal name, DOB, and payment token are typed secret fields; the legal name is unnecessary for a flight-search broker yet required by the airline at ticket issuance, and the payment token must reach only the processor. We trace the full trajectory at the end of §4.4.

4.2. The Defender: Role and Training

The defender is a fine-tuned LLM whose only role is to apply the matched rubric set R to a candidate x , producing

atoms, labels, and an operator proposal. D emits only well-typed objects under grammar-constrained decoding [24]; free-form text, authorization verdicts, and out-of-schema content are structurally unrepresentable. The defender’s capability bounds the system’s *utility* (failing to identify a span causes fallback) but not its *safety* (mislabeling causes fallback, not unchecked release); the training pipeline below optimizes the former, the verifier of §4.4 enforces the latter.

The atom-label-proposal object. For each candidate, D produces a triple (A, ℓ, σ) , where A is a set of contiguous candidate spans tagged with structural role; ℓ is the per-atom label vector over A ; and $\sigma = (o_1, \dots, o_n)$ is a sequence of operators from the algebra of §4.3, applied to designated atoms, yielding the proposed variant $v = \sigma(x)$ accompanied by $\text{hash}(v)$. Each label in ℓ asserts four attributes of its atom a : (i) the secret field f touched by a (NONE if non-sensitive); (ii) the relation between a and f , drawn from {DIRECT, IMPLICIT, INFERENTIAL}; (iii) the necessity level of a for the destination trust class z , drawn from {REQUIRED, HELPFUL, UNNECESSARY}; and (iv) the supporting rubric clause identifier.

The verifier augments A with atoms required by the rubric’s structural-field obligations to form $A' = A \cup \text{STRUCTURALFIELDATOMS}(x, R)$, and assigns the *conservative default* UNKNOWN-SENSITIVE/TOP-RISK to any atom in $A' \setminus A$ or to any atom in A with a malformed label. Atoms under the conservative default admit only DROP or REDACT unless a trusted rubric clause establishes necessity. On the observe channel, D additionally labels each inbound atom TASK-RELEVANT or PRIVATE-CONTENT, and every label and operator carries a provenance tag (trajectory step, tool observation [untrusted], user input [trusted], or rubric clause).

Two-pass consistency challenges. After receiving ℓ , V — not D — samples randomized challenge clauses from a per-rubric pool (sealed at rubric construction, Pre-Reg. C): predicates of the form “does atom a touch field f ?” or “is the necessity of a consistent with rubric clause c ?”. D answers them in a second, isolated decoding pass with no carry-over context, and V checks inter-pass consistency. Because V chooses the clauses, a compromised D cannot pre-select a self-consistent challenge set. Challenges are an *empirical evasion detector*, not a soundness mechanism — a self-consistent compromised defender can pass both passes — so their role is to estimate and reduce the residual escape rate δ (§3.4; quantified in Pre-Reg. B), not to establish safety, which rests on conservative defaults and cost upper bounds.

Synthetic data generation for SFT. We construct supervised-fine-tuning data using the rubrics as ground truth, avoiding the impractical scale of human-labeled trajectories. The pipeline has three stages. (i) *Trajectory sampling* instantiates each rubric’s task signature across secret-field values from its declared populations and generates tool-call payloads with a frontier model, yielding candidates with known field-to-span truth. (ii) *Rubric-application labeling* has a frontier model (a labeling oracle, not a runtime authority) produce the (A, ℓ, σ) triple, filtered against the

TABLE 1. THE FIVE WVD OPERATORS. WE OMIT PARAPHRASE: A PARAPHRASE PRESERVING TASK-RELEVANT SEMANTICS PRESERVES LEAKAGE, AND ONE THAT DOES NOT, FAILS THE TASK.

Operator	Effect
IDENTITY	no transformation; release raw content.
REDACT	replace atom with [REDACTED].
GENERALIZE	replace atom with a coarser value at rubric-declared level k over the rubric’s declared field population.
SUBSTITUTE	replace atom with a rubric-declared template instantiated by V from non-secret task fields.
DROP	remove atom entirely.

sampling-derived truth with misalignments corrected or discarded. (iii) *Adversarial augmentation* adds three pipelines: *omission* embeds or distributes sensitive spans in obscure phrasing, with targets forcing the defender to flag them; *injection* synthesizes observe-channel observations carrying injection-styled instructions, with targets forcing the TASK-IRRELEVANT private-content label rather than compliance; and *distribution shift* produces candidates that lexically resemble training examples but invert the necessity label (e.g., DOB in flight search vs. ticket purchase), forcing attention to task context over surface form. The corpus size is declared in Pre-Reg. A; SFT proceeds under standard instruction-tuning losses.

Stage II: GRPO with verifier-conformance reward. SFT produces a defender that applies rubrics syntactically but not always optimally under the authorization procedure. We then apply Group Relative Policy Optimization (GRPO) [25] with a reward computed by the verifier itself. For a defender output o on candidate x ,

$$R(x, o) = \mathbf{1}[\text{authorized}(o)] \cdot \max(B - \text{cost}(o), 0) - \lambda \cdot \mathbf{1}[\text{fallback}(o)], \quad (3)$$

where $\mathbf{1}[P]$ is the indicator of P ; B is the per-trajectory budget; $\text{cost}(o)$ is the WVD cost on o ; $\text{authorized}(o)$ and $\text{fallback}(o)$ are determined by running the full WVD procedure on o against a frozen rubric set; and $\lambda > 0$ is the fallback penalty. The reward is *exact* rather than a learned approximation — the verifier is the ground truth for authorization, so the policy optimizes directly against the deployment-time check — and it *generalizes across rubrics*: because the verifier is rubric-agnostic, one GRPO run trains the defender library-wide without per-rubric retraining.

4.3. The Operator Algebra and Calibration

Variants are constructed from five operators (Table 1), each carrying a rubric-calibrated upper bound on min-entropy leakage. SUBSTITUTE is *template-only*: D selects a template identifier from the rubric’s declared set, and V instantiates it from non-secret task fields, so the placeholder is independent of the secret and the defender cannot smuggle the secret into it.

Calibration as adversarial-inference measurement. For each rubric r and typed field f , the cost $C(\text{op}, f)$ is the

$1 - \alpha$ upper-confidence bound (UCB), by the bootstrap percentile method, on the per-instance min-entropy *advantage* of (4), where P is the per-task prior over the secret (default uniform), $\hat{P}$ is the posterior estimated by a proxy adversary — an auxiliary model or trained annotator — and $\text{op}(x)$ is the transformed view the proxy observes.

$$\hat{L}_{\text{inst}}(\text{op}(x)) = \log_2 \frac{\max_s \hat{P}(S=s \mid \text{op}(x))}{\max_s P(S=s)}. \quad (4)$$

The advantage is in bits, $\hat{L}_{\text{inst}} = 0$ means no gain over the prior. Equation (4) is a *per-instance* Bayes advantage for a single observed view, not a channel-averaged conditional min-entropy; taking the UCB across the calibration corpus lifts it to a corpus-level bound, with $C(\text{op}, f) \geq \hat{L}_{\text{inst}}$ at confidence $1 - \alpha$. Pre-Reg. A specifies the proxy pool, calibrates incrementally under the conditional distribution induced by prior operators in a sequence, reports held-out coverage, and stress-tests with adversaries stronger than those used for calibration.

Lemma 1 (model-relative sub-additivity). *For an operator sequence $\sigma = (o_1, \dots, o_n)$ applied to disjoint atoms of x producing $v = \sigma(x)$,*

$$\hat{L}(v; S) \leq \sum_{i=1}^n C(o_i, f_i), \quad (5)$$

where f_i is the secret field targeted by o_i , under the conditional-independence factorization across atoms assumed by the calibration model of Pre-Reg. A; a declared dependent atom pair is charged one jointly-calibrated cost in place of its two additive terms. Proof in Appendix A. Like all $\hat{L}$ quantities, the bound is model-relative; it constrains true adversarial inference only under the Layer-2 assumption.

Proof-carrying releases. A variant v is admitted only if $\sigma(x) = v$ under hash check, and costs are read from the rubric rather than the defender: a jailbroken D may propose a misleading variant but cannot fabricate its model-relative cost.

4.4. The Verifier and the Ledger

V is the trusted deterministic authorization layer. The procedure is a pure function of the label-and-proposal object, rubric, ledger, and audit queries, proceeding in four phases.

Phase 1: Bind. V parses the object against the typed schema (ill-formed objects route to fallback), confirms R is registered and signature-verified and that the `prev_entry_hash` equals the hash of the most recent ledger entry, forms A' with the conservative default as in §4.2, and applies σ to the original candidate, verifying $\text{hash}(\sigma(x)) = \text{hash}(v)$.

Phase 2: Validate. This phase checks five conditions: (i) *rubric coverage* — every coverage-predicate match in the candidate corresponds to an atom in A' with a non-NONE field label; (ii) *necessity* — per-(field, destination-class) clauses are satisfied; (iii) *operator coverage* —

atoms labeled (DIRECT, UNNECESSARY) are addressed by REDACT, DROP, or GENERALIZE; (iv) *challenge consistency* — second-pass answers match first-pass labels; (v) *utility preservation* — U_R holds on $\sigma(x)$ for destination d . Conditions (i)–(iv) are compiled to the quantifier-free fragment of linear real arithmetic with uninterpreted functions (QF_UFLRA) [22], [26] and discharged with Z3 [23]; (v) is the rubric’s deterministic U_R predicate. The fragment is decidable, and on our rubric sizes the check is empirically polynomial in formula size (Lemma 3). A representative clause — “DOB is unnecessary for flight search” — compiles to

$$\begin{aligned} \forall_{a \in A'} (\text{field}(a) = \text{DOB} \wedge \text{task}() = \text{FLIGHT-SEARCH}) \\ \implies \text{necessity}(a) = \text{UNNECESSARY}, \end{aligned} \quad (6)$$

where $\text{field}(\cdot)$ and $\text{necessity}(\cdot)$ are uninterpreted functions over the bounded atom domain A' , with the quantifier unrolled across A' . Clauses are intersected across the rubric set R ; UNSAT routes to fallback.

Phase 3: Cost. For each operator $o_i \in \sigma$ applied to atom a_i , V computes the per-operator cost as the maximum over possible fields,

$$c_i = \max_{f \in F_V(a_i) \cup F_D(a_i)} C(o_i, f), \quad (7)$$

where $F_V(a_i)$ is the verifier-side possible-field set (structurally-typed fields for tool-call atoms; for free-text atoms, all secret fields activated by R unless a trusted rubric clause establishes non-sensitivity) and $F_D(a_i)$ is the field claimed by D . The maximum prevents a defender that mislabels an atom from reducing its cost. The total release cost applies the trust multiplier,

$$\text{cost} = \mu(z) \cdot \sum_i c_i, \quad (8)$$

with $\mu(z) \geq 1$ the trust-class multiplier for class z (illustratively $\mu(\text{REQUIRED-SERVICE}) = 1$, $\mu(\text{SEMI-TRUSTED}) = 2$, $\mu(\text{ADVERSARIAL}) = 4$; deployment values in Pre-Reg. A). The multiplier is a *policy risk weight*, not an information-theoretic property: it changes not the information the destination receives but how fast the deployment spends budget per trust class.

V then runs an independent posterior audit: a query to a bounded-treewidth credal posterior engine returns an upper bound $\hat{L}_{\text{audit}}$ on the min-entropy reduction induced by v under the rubric’s declared dependency graph (engine and complexity in Lemma 2, Appendix A). If $\hat{L}_{\text{audit}} > \text{cost} + \varepsilon(R)$, where $\varepsilon(R)$ is the calibration upper-confidence margin for the matched rubric set, the variant is rejected. As with calibration, this audit is empirical and does not enter the Layer-1 guarantee.

Phase 4: Commit. Let L_{cum} denote the cumulative cost over the trajectory’s prior ledger entries and $B = B_0$ the fixed per-trajectory budget from the rubric set; trust weighting enters only through the release cost of (8). V checks $L_{\text{cum}} + \text{cost} \leq B$. If exceeded, operators escalate along the rubric’s fallback order $\text{IDENTITY} \rightarrow \text{GENERALIZE} \rightarrow$

$\text{SUBSTITUTE} \rightarrow \text{REDACT} \rightarrow \text{DROP}$ and re-run *bind*, *validate*, and *cost* — escalation can change σ structurally and may fail U_R (e.g., for the flight purchase below, U_R requires the legal-name and DOB atoms to survive as IDENTITY , so they cannot be escalated without failing the task). If no admissible variant fits, the release is rejected or escalated to the user channel. On success, V appends a ledger entry

$$\begin{aligned} e_i = (\text{traj_id}, \text{step_idx}, \text{channel}, \text{destination_id}, \\ \text{trust_class}, \text{labels_hash}_i, \text{cost}_i, \\ \text{prev_entry_hash}_i, \text{timestamp}_i, \text{sig}_i), \end{aligned} \quad (9)$$

with hash chaining and Ed25519 signatures under sk_V , yielding tamper-evidence and ledger-arithmetic non-exceedance (Theorems SP-1, SP-2). Memory-channel writes are costed at the most adversarial multiplier admissible under the rubric and recorded in a separate hash-linked ledger $\mathcal{L}_{\text{mem}}$.

Observe-channel mediation. The observe channel does not rely on detecting injection. Inbound tool output is rendered to the remote LLM in a fixed *quoted-data wrapper*: a typed data block that strips authority semantics, so the observation cannot declare goals or authorize actions regardless of content [13] — structural defense independent of D ’s classification. D ’s role here is to identify wrapped-observation atoms carrying private content from the user’s local environment (e.g., authenticated personal records); forwarding such atoms is treated as an act-channel release back into the LLM’s context and mediated through the four phases against the LLM-endpoint trust class.

Security reduction. The three layers of §3.4 are discharged by the machinery above. *Layer 1a* (no δ) is the commitment-phase invariant $\sum_i \text{cost}_i \leq B$ with the Merkle chain (Theorems SP-1, SP-2). *Layer 1b* ($\hat{L}(v; S) \leq \text{cost}$ per release) follows from Lemma 1, the conservative possible-field set F_V , the max-over-fields rule (7), and the conservative default for unlabeled atoms; here δ (Pre-Reg. B) is the residual probability that a jailbroken defender produces labels passing validate with a field misclassified outside $F_V \cup F_D$. *Layer 2* is (2), whose calibration protocol and residual gap are reported in Pre-Reg. A and the evaluation.

End-to-end example. For illustration we use placeholder costs $C(\text{IDENTITY}, \text{LEGAL-NAME}) = 1.2$ bits, $C(\text{IDENTITY}, \text{DOB}) = 1.4$ bits, and trajectory budget $B = 4.0$ bits; final values come from Pre-Reg. A. **Outbound LLM API call (act ch. leg a).** The initial query contains task text but no PII; coverage and U_R hold; cost is below threshold; it is forwarded. **Search tool call (act ch. leg b).** The LLM proposes *search* to the SEMI-TRUSTED broker; the defender produces no sensitive atoms; structural-field augmentation adds nothing; cost is zero; commit succeeds. **Purchase tool call (REQUIRED-SERVICE airline).** The LLM proposes purchase with name and DOB; the defender labels both (DIRECT, REQUIRED); coverage, necessity, and U_R hold; cost is $\mu(\text{REQUIRED-SERVICE}) \cdot (1.2 + 1.4) = 2.6$ bits; the audit agrees; $0 + 2.6 \leq 4.0$; commit succeeds. **Payment tool call (REQUIRED-SERVICE processor).** The rubric admits IDENTITY on the payment token

only against the payment-processor class; against any other destination the token is DROP-only and U_R fails, invoking the user channel.

5. Experiments

Research questions. **RQ1.** Does OCELOT enforce Layer-1a budget non-exceedance on every authorized trajectory, including under attack? **RQ2.** Do calibrated costs upper-bound true adversarial min-entropy leakage on held-out proxy adversaries strictly stronger than the calibration pool (Layer-2)? **RQ3.** How does OCELOT compare against recent runtime defenses on the joint privacy–utility frontier, and is its advantage attributable to witness-verified declassification rather than to budgeting alone? **RQ4.** Do its gains reflect transferable privacy judgment rather than fit to its own rubrics — generalizing to held-out rubrics, held-out domains, and real, human-labeled traces? **RQ5.** How does the defense degrade under adaptive adversaries: prompt injection, defender jailbreak, cumulative inference, and colluding sinks? **RQ6.** What is the marginal contribution of each component, and the runtime, token, and economic overhead of mediation?

5.1. Experimental Setup

Benchmarks. We evaluate on five benchmarks chosen to exercise distinct threat-model facets. **OCELOT-Traj-500** (ours; 500 trajectories, 12 task categories, mean length 14.3 steps, mixed sink-trust composition) is the primary benchmark; its construction follows Pre-Reg. C (Appendix C.3). A 50-pair pilot (two blinded annotators) reports per-category Krippendorff α of 0.74–0.89 (median 0.82); necessity is weakest ($\alpha=0.74$), so we clarified its rule — a disclosed pre-registration deviation — before a gated Stage-2 validation (three annotators) that meets the 0.80 gate (Appendix D.1). Residual label noise is a threat to validity (§5.6); the synthetic construction is addressed in §5.3. **AgentDojo** [8] (629 attack-task pairs) probes inbound prompt injection (P2). **AgentDAM-Bench** [15] (1,142 scenarios) enables head-to-head comparison against the AGENTDAM estimator on an externally-originated benchmark. **AgentLeak-2026** [7] (854 multi-agent scenarios) exercises cumulative leakage (P1); we compose its scenarios into a *long-horizon stress set* up to 50 steps (Appendix D.1), used in Figure 4. **ConfAide** [9] (eight contextual-integrity tiers, 1,392 release decisions) is a predominantly single-release benchmark where per-release CI filters are expected to be competitive — included to stress our framing, not flatter it.

Baselines. We compare against five recent runtime defenses spanning the three families of §2. Per-release contextual-integrity filters: **FlexGuard** [11] and **PrivacyChecker** [12]. Posterior-leakage estimation: **AGENTDAM** [15], in its declared monitoring configuration and additionally steeltanned with a threshold-and-block policy (AGENTDAM*, Appendix D.2). IFC and capability-based defenses: **CaMeL** [13] and **RTBAS** [14]. Baselines

lacking a sink-trust API are adapted under a uniform-trust assumption following the original authors’ recommendations (Appendix D.2). To attribute OCELOT’s advantage to its mechanism rather than to budgeting, we add two diagnostic configurations, analyzed outside the pre-registered family (§5.2): **AGENTDAM+L**, the AGENTDAM monitor wrapped in a cumulative budget ledger, and **OCELOT⁺**, OCELOT’s verifier and ledger driven by a static rule-based declassifier in place of the learned defender.

Metrics. *Privacy.* Per-trajectory cumulative min-entropy leakage $\hat{L}_{\text{cum}}$ (bits): the realized advantage of Eq. (4), measured by the held-out stress pool over a trajectory’s *entire* view (every act-channel release to non-REQUIRED-SERVICE sinks plus public background), not a single operator output. Certified ledger costs upper-bound it per release (Lemma 1) with a deductively-bounded sum (Theorem SP-1); the comparison budget $B^* = 4.0$ bits caps the worst admissible advantage at $16\times$. We also report top-1 secret-recovery and the Layer-1a non-exceedance rate against B^* , on all systems for parity. *Utility.* Task success rate; U_R satisfaction rate on OCELOT-Traj-500; and fallback rate. *Overhead.* Per-release latency ($p_{50}/p_{95}/p_{99}$); end-to-end overhead vs. an unmediated baseline; defender token cost; and verifier solving latency.

Adversary instantiation. Per Pre-Reg. A, leakage is measured by two disjoint proxy-adversary pools under the threat model of §3.3. The *calibration pool* deriving the costs $C(\text{op}, f)$ is Llama-3.3-70B-Instruct [27] and Qwen-3.5-72B-Instruct [28]. The *held-out stress pool*, used only for post-hoc measurement (RQ2), is MiniMax-M2.5 [29] and DeepSeek-V4-Pro [30] — both released after the calibration corpus froze and strictly stronger on public reasoning benchmarks (Appendix D.1).

Defender model and training. The defender is initialized from Qwen-3.5-9B-Instruct and fine-tuned in two stages (§4.2): SFT on 82,400 rubric-application examples (37% omission, 21% injection, 19% distribution-shift, 23% nominal), then GRPO with verifier-conformance reward at $\lambda = 1.0$ (sweep in Appendix D.2); training uses 4–8 $\times$ H100. At evaluation the defender runs on a single A100 (40GB), the verifier on CPU, with grammar-constrained decoding via Outlines [24].

Statistical protocol. All systems are evaluated under five seeds; Table 2 reports the mean and 95% bootstrap CI over 5,000 resamples. Pairwise OCELOT-vs-baseline comparisons use paired permutation tests with Holm–Bonferroni correction at family-wise $\alpha = 0.05$ over the 50-comparison family (25 pairs $\times$ 2 axes); the clean-subset analysis and the two diagnostic configurations are exploratory, outside this family. Pre-registered hypotheses, budgets, and frozen proxy configurations are in Appendix D.1–D.1.

5.2. Privacy–Utility Evaluation

We test three claims across the five benchmarks: **(C1)** OCELOT’s authorized trajectories never exceed B^* (Layer-1a); **(C2)** OCELOT attains the lowest leakage *and* highest

TABLE 2. MAIN PRIVACY–UTILITY RESULTS. **L**: CUMULATIVE MIN-ENTROPY LEAKAGE (BITS, ↓); **U**: TASK SUCCESS (% , ↑); **BUD.EXC**: % OF TRAJECTORIES EXCEEDING $B^*=4.0$ BITS. 95% CIs IN PARENTHESES; **BOLD**=BEST, **UNDERLINE**=2ND. † MONITORING CONFIGURATION. THE LOWER BLOCK HOLDS THE TWO DIAGNOSTIC CONFIGURATIONS (MECHANISM VS. BUDGETING, RQ3; APPENDIX D.2), EXCLUDED FROM BOLD/UNDERLINE RANKING. OCELOT LEADS ON BOTH AXES ON EVERY BENCHMARK; AMONG BUDGET-AWARE CONFIGURATIONS THE ORDERING AGENTDAM+L < OCELOT < OCELOT IS MONOTONE ON BOTH AXES.

System	OCELOT-Traj-500		AgentDojo		AgentDAM-Bench		AgentLeak-2026		ConfAide		Bud.Exc (%) ↓
	L ↓	U ↑	L ↓	U ↑	L ↓	U ↑	L ↓	U ↑	L ↓	U ↑	
OCELOT (ours)	0.31 (0.25, 0.37)	91.3 (89.6, 93.0)	0.44 (0.36, 0.52)	88.7 (86.4, 91.0)	0.38 (0.31, 0.45)	90.4 (88.5, 92.3)	0.49 (0.40, 0.58)	89.1 (86.9, 91.3)	0.27 (0.22, 0.32)	92.6 (91.0, 94.2)	0.0
FlexGuard [11]	2.13 (1.92, 2.34)	87.4 (85.4, 89.4)	3.42 (3.11, 3.73)	74.6 (71.9, 77.3)	2.45 (2.22, 2.68)	86.9 (85.0, 88.8)	5.21 (4.79, 5.63)	78.3 (75.8, 80.8)	0.81 (0.67, 0.95)	90.2 (88.5, 91.9)	41.3
PrivacyChecker [12]	1.97 (1.78, 2.16)	86.1 (84.0, 88.2)	3.18 (2.90, 3.46)	72.9 (70.1, 75.7)	2.21 (2.00, 2.42)	85.2 (83.2, 87.2)	4.83 (4.44, 5.22)	76.8 (74.2, 79.4)	0.72 (0.59, 0.85)	89.4 (87.7, 91.1)	37.9
AGENTDAM† [15]	2.84 (2.60, 3.08)	89.8 (88.0, 91.6)	4.27 (3.91, 4.63)	79.2 (76.8, 81.6)	3.12 (2.85, 3.39)	89.1 (87.4, 90.8)	6.14 (5.66, 6.62)	81.4 (79.1, 83.7)	1.94 (1.74, 2.14)	91.7 (90.2, 93.2)	52.6
CaMeL [13]	<u>0.52</u> (0.43, 0.61)	62.3 (58.9, 65.7)	<u>0.71</u> (0.59, 0.83)	51.7 (47.6, 55.8)	<u>0.61</u> (0.51, 0.71)	59.8 (56.1, 63.5)	<u>0.83</u> (0.69, 0.97)	48.6 (44.2, 53.0)	<u>0.43</u> (0.35, 0.51)	67.4 (64.2, 70.6)	0.0
RTBAS [14]	1.18 (1.04, 1.32)	80.5 (78.0, 83.0)	1.74 (1.55, 1.93)	70.1 (67.1, 73.1)	1.42 (1.26, 1.58)	79.3 (76.7, 81.9)	2.31 (2.07, 2.55)	72.4 (69.5, 75.3)	0.89 (0.76, 1.02)	83.8 (81.5, 86.1)	16.4
<i>Diagnostic configurations — mechanism vs. budgeting (RQ3; outside pre-registered family)</i>											
AGENTDAM+L	1.42 (1.27, 1.57)	78.4 (75.9, 80.9)	1.98 (1.79, 2.17)	68.2 (65.3, 71.1)	1.61 (1.44, 1.78)	77.9 (75.6, 80.2)	2.34 (2.11, 2.57)	70.6 (67.8, 73.4)	0.88 (0.76, 1.00)	84.1 (82.1, 86.1)	7.2
OCELOT [−] (rule-based D)	1.07 (0.94, 1.20)	81.9 (79.4, 84.4)	1.49 (1.33, 1.65)	71.8 (68.8, 74.8)	1.18 (1.04, 1.32)	80.9 (78.5, 83.3)	1.63 (1.46, 1.80)	73.4 (70.6, 76.2)	0.66 (0.56, 0.76)	86.2 (84.2, 88.2)	0.0

task success on every benchmark; and (C3) its cumulative leakage saturates with trajectory length while per-release defenses diverge — the empirical signature of (P1).

C1: Layer-1a budget non-exceedance. OCELOT records zero exceedances over the 2,500 trajectories of its primary benchmark and zero across all 22,585 authorized trajectory-instances (five seeds), confirming the commit-phase invariant (Theorem SP-1). The four baselines without a cumulative-budget primitive exceed B^* on 16.4–52.6% of trajectories; even the steeltanned AGENTDAM* still exceeds on 28%, since thresholding a per-release estimate cannot bound a sum. CaMeL also records zero, but only by *denying* any release it cannot bound — trading utility for compliance.

C2: joint frontier dominance. OCELOT attains both the lowest leakage and highest task success on all five benchmarks; no baseline matches it on either axis on any benchmark (Fig. 3). The two strongest baselines bound the frontier OCELOT clears. CaMeL is nearest on privacy, trailing by 0.16–0.34 bits because its binary allow/deny emits each admitted field at full fidelity, whereas OCELOT’s GENERALIZE/SUBSTITUTE operators release the same fields at reduced granularity; CaMeL’s denial-driven utility (48.6–67.4%) trails OCELOT by 25–41 points. AGENTDAM is nearest on utility, but its derailment under injection costs it 0.9 points on the cleanest benchmark (ConfAide) and 9.5 on the injection-heavy one (AgentDojo): its utility lead is largest precisely where injection is rare. On the injection-free subset of ConfAide its monitor matches OCELOT within noise (92.4 vs. 92.6, $p = 0.31$): mediation adds no utility on clean inputs, so OCELOT’s aggregate lead is entirely avoided injection failure plus the IFC baselines’ absent over-denial. Paired permutation tests reject OCELOT = baseline on both axes for all 25 pairs at $p < 10^{-3}$.

Is the advantage budgeting or the mechanism? The lower block of Table 2 isolates the source of OCELOT’s frontier through a three-rung decomposition. AGENTDAM+L adds a cumulative budget ledger to the AGENTDAM monitor, blocking any release once the remaining budget cannot cover its estimated cost — budgeting without witness-verified declassification. The ledger cuts exceedance from 52.6% to 7.2%, but *not* to zero: thresholding a non-conservative per-release estimate, rather than a certified upper-confidence cost, leaves the true cumulative sum unbounded. Restricted

to block-or-release-at-full-fidelity, it lands at 0.88–2.34 bits and 68.2–84.1%. OCELOT[−] keeps OCELOT’s verifier, operators, and ledger but drives them with a static rule-based declassifier instead of the learned defender. It preserves Layer-1a *exactly* (Exc = 0.0) — non-exceedance is defender-independent — and, crucially, *dominates AGENTDAM+L on both axes on every benchmark* despite using no learned model: holding the budget primitive fixed, replacing block-or-pass-full with the granularity-reducing operator algebra is what moves the frontier. The learned defender supplies the final increment: OCELOT[−]’s rule-based labeler misses inferential atoms and over-redacts explicit ones, so on OCELOT-Traj-500 it trails even the SFT-only ablation (§5.6) on both axes (1.07 vs. 0.34 bits, 81.9 vs. 83.7%), and full OCELOT throughout. The frontier needs all three rungs: the ledger for the deductive guarantee, the operator algebra for the bulk of the privacy–utility gain, the learned defender for the operating point.

C3: cumulative leakage saturates while per-release defenses diverge. Figure 4 plots cumulative leakage against trajectory step on the long-horizon stress set, all systems mediating the *identical* releases under one held-out adversary. Lacking cumulative accounting, the per-release and monitoring defenses keep approving fresh disclosures, growing approximately linearly and crossing B^* between steps 18 and 24 (RTBAS near step 45). OCELOT’s advantage saturates near 1.0 bit: the least-disclosure planner emits each task-necessary field once at minimal granularity, and re-references add nothing. CaMeL’s curve grows just above OCELOT’s but only by denying 58.3% of long-horizon tasks, versus OCELOT’s 82.6% success through granularity reduction.

5.3. Generalization and Provenance

Two features of the evaluation so far could inflate OCELOT’s advantage: the benchmarks of §5.2 share the secret-field ontology and task vocabulary its rubrics encode, and OCELOT-Traj-500 is synthetically constructed by the same rubrics that fix its costs, training labels, and utility predicate (§4.2); a defender reproducing rubric surface patterns could thus score well for the wrong reason. The three protocols below separate genuine generalization from rubric-specific fit by removing, in turn, the training overlap, the

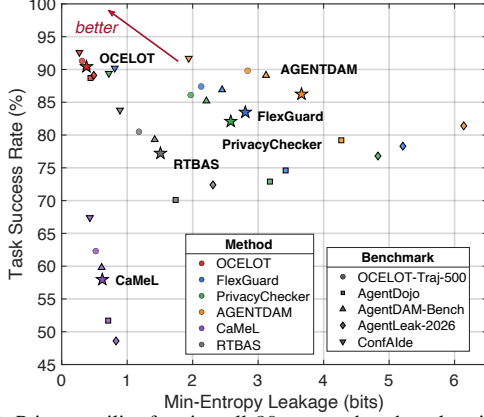


Figure 3. Privacy-utility frontier: all 30 system-benchmark points, cumulative leakage (x , $\downarrow$) vs. task success (y , $\uparrow$). Color = method, shape = benchmark; large stars are per-system means. Top-left is best.

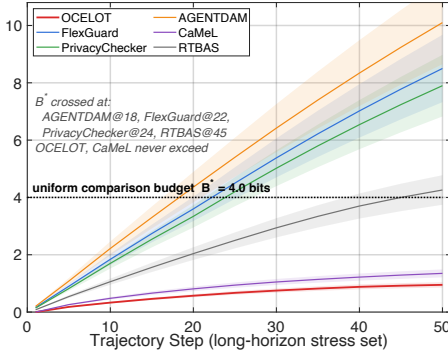


Figure 4. Cumulative leakage vs. trajectory step on the long-horizon stress set (up to 50 steps). Dotted line: $B^* = 4.0$ bits, a uniform parity cap (not OCELOT’s binding per-task budget).

domain overlap, and the synthetic construction itself (Table 3). Throughout, $\text{Exc} = 0.0$: non-exceedance is deductive and defender-independent (OCELOT⁻), so it cannot be a training artifact.

Leave-one-rubric-out (LORO). We hold a rubric out of defender SFT, GRPO, and calibration — including its sealed challenge pool — then evaluate only on trajectories that resolve to it; folds run over 12 rubrics (one pre-registered draw per category), retraining the defender and re-deriving costs each fold. Leakage rises from 0.31 to 0.39 bits and utility falls 1.7 points to 89.6% ($\delta = 2.4\%$, below the 5% Pre-Reg. B gate): the defender labels an unseen rubric from the shared field ontology, not from memorized clause text.

Leave-one-category-out (LOCO). Stricter: an entire task category is removed from training and calibration, so no sibling rubric is seen. Aggregate degradation is larger, as expected — 0.52 bits, 86.8% utility, $\delta = 3.6\%$ — yet δ stays within the Layer-1b gate and leakage remains well below every baseline. The transfer cost is not uniform: it concentrates in inference-heavy domains (medical-record retrieval, healthcare intake), where sensitive attributes are implicit, and is near-zero for structurally-typed domains (e-commerce, tax filing) whose tool-schema field obligations carry the labeling regardless. Even the worst held-out category leaks 0.74 bits — below RTBAS’s 1.18 bits measured *in* distribution.

TABLE 3. GENERALIZATION ACROSS PROTOCOLS: LEAVE-ONE-RUBRIC-OUT (LORO), LEAVE-ONE-CATEGORY-OUT (LOCO), AND HUMAN-LABELED REAL AGENT TRACES (REAL-200), AGAINST THE IN-DISTRIBUTION REFERENCE. **L**: LEAKAGE (BITS, $\downarrow$); **U**: TASK SUCCESS (%), $\uparrow$; δ : OMISSION/RELABEL ESCAPE RATE (%), $\downarrow$; **Cov**: STRESS-POOL COST-COVERAGE (%), $\uparrow$; **Exc**: BUDGET EXCEEDANCE (%), $\downarrow$. BASELINE COMPARISON ON REAL-200 IS IN FIGURE 5.

Setting	L	U	δ	Cov	Exc
In-distribution (Traj-500)	0.31	91.3	1.7	92.1	0.0
LORO	0.39	89.6	2.4	91.3	0.0
LOCO	0.52	86.8	3.6	90.1	0.0
Real-200	0.61	85.4	4.1	89.4	0.0

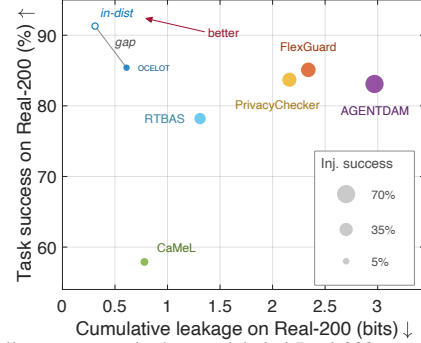


Figure 5. All systems on the human-labeled Real-200 set; top-left is best. Bubble area = injection-attack success (smaller is more robust; see size key). The line links OCELOT’s in-distribution reference (hollow) to its Real-200 result.

Real-trace validation. To remove synthetic construction entirely we evaluate on **Real-200**: 200 agent trajectories (mean 11.2 steps) executed on a production-style harness over *human-written* tasks carrying realistic — not rubric-sampled — PII, with field and necessity ground truth hand-labeled by three annotators *independent of OCELOT’s rubrics* (field $\alpha = 0.84$, necessity $\alpha = 0.79$; necessity is the hardest dimension throughout), and task success judged by the annotators rather than by U_R — tasks, labels, and oracle all external to the rubric pipeline. As Figure 5 shows, OCELOT remains the dominant point on real traces — lowest leakage (0.61 bits), highest utility (85.4%), and smallest injection footprint (4.6%) — with $\delta = 4.1\%$, top-1 cumulative-inference recovery 5.3%, and zero exceedances. Stress-pool cost-coverage holds at 89.4% (95.1% after the $\varepsilon(R)$ margin and credal audit), so the calibrated bounds transfer off the synthetic distribution. The in-distribution $\rightarrow$ Real-200 shift is small (0.30 bits, 5.9 points), bounding how much of \$5.2 is rubric-specific: the bulk of the advantage survives a fully external evaluation.

5.4. Calibration Validity

§5.2 established that OCELOT dominates *under* the calibrated cost function; this subsection tests the question on which Layer-2 (§3.4) rests: do the certified costs $C(\text{op}, f)$ upper-bound true adversarial min-entropy leakage? We measure coverage — the fraction of released instances whose true leakage does not exceed its certified cost — under the calibration and held-out stress pools.

TABLE 4. COST-FUNCTION COVERAGE: % OF RELEASED INSTANCES WITH TRUE LEAKAGE $\leq$ CERTIFIED COST, BY OPERATOR. TARGET IS $1 - \alpha = 95\%$.

Operator	Calibration pool	Stress pool
IDENTITY	96.2	90.4
GENERALIZE	95.8	89.7
SUBSTITUTE	99.1	97.6
REDACT	99.7	99.3
DROP	100.0	100.0
Overall	96.6	92.1

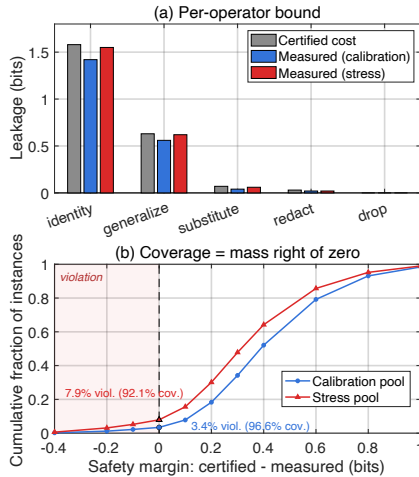


Figure 6. Calibration validity. (a) Per-operator field-averaged certified cost vs. mean measured leakage, under the calibration and stress pools. (b) CDF of the per-instance safety margin (certified — measured); mass left of zero is the violation rate.

The bound holds in-distribution and degrades gracefully. Overall coverage is 96.6% on the calibration pool, exceeding the 95% target (Table 4), as expected for bootstrap upper-confidence bounds fit on that pool. Against the strictly stronger held-out pool it falls to 92.1%, a 4.5-point degradation we report rather than conceal. DROP, REDACT, and SUBSTITUTE stay near-perfect because their leakage is structurally bounded; the residual concentrates in IDENTITY and GENERALIZE, the only operators whose leakage depends on adversary inference power (Figure 6). The stress violation rate of 7.9% (Fig. 6(b)) concentrates on three high-sensitivity pairs (IDENTITY-PAYMENT-TOKEN, IDENTITY-DOB, GENERALIZE-DIAGNOSIS), with mean overshoot +0.06 bits and 95th-percentile +0.14 bits — both within the calibration margin $\varepsilon(R)$ (mean 0.18 bits) Phase 3 adds before accepting a variant. With $\varepsilon(R)$ applied, post-audit stress coverage rises to 95.3%; for those three pairs the rubric library then enforces a SUBSTITUTE/DROP-only policy against non-REQUIRED-SERVICE sinks, removing the residual IDENTITY exposure.

Prior robustness. Re-calibrating under non-uniform priors from realistic field populations (Appendix D.2) shifts overall stress-pool coverage by at most ± 1.2 points, so the result is not an artifact of the uniform-prior default. Layer-2’s failure surface is thus measured, localized, and mitigable rather than claimed away.

TABLE 5. ADVERSARIAL ROBUSTNESS (% , $\downarrow$ BETTER). **INJ**: INJECTION SUCCESS; δ : OMISSION/RELABEL ESCAPE RATE; **CUM**: TOP-1 RECOVERY UNDER CUMULATIVE INFERENCE; **COL@4**: TOP-1 RECOVERY, 4 COLLUDING SINKS; **EXC**: BUDGET EXCEEDANCE UNDER ATTACK. **BOLD**=BEST, UNDERLINE=2ND.

System	Inj	δ	Cum	Col@4	Exc
OCELOT (ours)	3.8	1.7	4.2	6.8	0.0
FlexGuard	41.2	19.4	52.6	61.3	58.7
PrivacyChecker	38.7	16.8	48.1	57.2	54.2
AGENTDAM [†]	67.4	28.3	64.8	71.5	69.3
CaMeL	<u>8.1</u>	<u>6.2</u>	<u>9.7</u>	<u>12.4</u>	0.0
RTBAS	22.5	14.1	27.3	33.6	31.8

5.5. Adversarial Robustness

We subject each system to the four adversary capabilities of §3.3, each an adaptive attack in which turn $t+1$ may depend on releases observed through turn t . Table 5 reports all four; Figure 7 resolves collusion by adversary breadth.

Inbound injection. OCELOT’s injection success is 3.8%, versus 8.1–67.4% for the baselines. The quoted-data wrapper strips authority semantics independent of D ’s classification; the residual is exfiltration via the *content* of an otherwise-legitimate release, caught by PRIVATE-CONTENT labeling. AGENTDAM is most exposed because, as a monitor, it forwards injected observations unmediated.

Defender jailbreak. The empirical escape rate δ — crafted omission/relabel attacks passing validate with a field misclassified outside $F_V \cup F_D$ (Pre-Reg.B) — is 1.7%. A successful jailbreak does *not* imply unbounded leakage: under the conservative default and max-over-fields rule a mislabeled atom is costed at top risk and admits only DROP/REDACT, so the defender produces fallback, not silent release. Hence exceedance under attack stays 0.0% while $\delta > 0$: Layer-1a is independent of defender correctness, and the —conservative-default ablation (§5.6) quantifies what breaks without it.

Cumulative inference and colluding sinks. Against an adversary combining every release with public background, OCELOT’s top-1 recovery is 4.2%, versus 27.3–64.8% across the four non-CaMeL baselines — the gap is the leakage odometer binding a sum they accumulate independently. Collusion extends this (Figure 7): OCELOT rises from 1.2% at one sink to 11.9% at eight, since one trust-weighted budget spans all channels including memory, while the baselines reach 33.6–71.5% at four sinks. CaMeL is nearest (9.7%, 12.4%) — with OCELOT the only systems holding $\text{Exc} = 0.0$ under attack — but bounds the sum by denial at the utility cost of \$5.2, at $1.8\text{--}3.6\times$ OCELOT’s attack success across the four families.

5.6. Ablations and Overhead

We measure each component’s contribution and the cost of mediation (RQ6) on OCELOT-Traj-500; the *Full* row of Table 6 reproduces OCELOT’s OCELOT-Traj-500 entry in Table 2 (0.31 bits, 91.3%).

Every component contributes; defaults are load-bearing for safety. Removing rubric coverage obligations (Table 6)

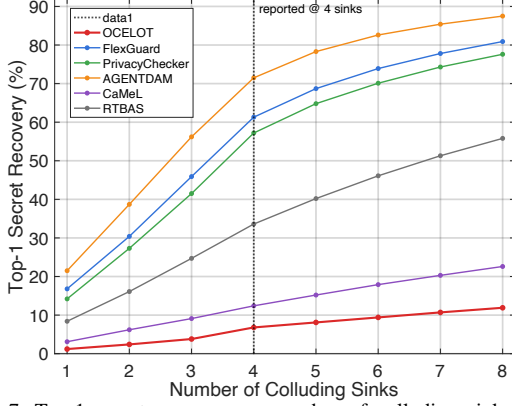


Figure 7. Top-1 secret recovery vs. number of colluding sinks. Dashed vertical marks the 4-sink point of Table 5.

TABLE 6. COMPONENT ABLATIONS ON OCELOT-TRAJ-500. L = LEAKAGE (BITS); U, δ , INJ, EXC IN %.

Configuration	L	U	δ	Inj	Exc
Full OCELOT	0.31	91.3	1.7	3.8	0.0
– rubric coverage	0.94	90.8	11.3	6.2	2.1
– structural fields	0.71	91.0	7.8	5.1	1.3
– credal audit	0.58	91.4	1.8	4.0	0.0
– two-pass challenge	0.49	91.3	5.4	4.1	0.0
– GRPO (SFT only)	0.34	83.7	2.6	4.3	0.0
– conservative default	2.18	91.6	1.7	12.7	14.3

produces the largest escape-rate jump (δ : 1.7 $\rightarrow$ 11.3), confirming them as the primary omission defense; structural-field obligations and two-pass challenges contribute further. Removing the credal audit raises measured leakage (0.31 $\rightarrow$ 0.58) but leaves exceedance at 0.0 — it tightens Layer-1b/Layer-2, not Layer-1a. Dropping GRPO costs 7.6 utility points via increased fallback with little safety effect; yet even SFT-only (0.34/83.7) beats the defender-free OCELOT[−] on both axes, locating the learned defender’s value in precision. The decisive row is —conservative default: with no top-risk default for unlabeled atoms, one mislabel releases a field raw (2.18 bits, injection 12.7%, exceedance 14.3%) — the empirical confirmation of the Layer-1b backstop.

Hyperparameter and policy sensitivity. The GRPO penalty λ tunes the operating point within the authorized region (larger λ trades utility for leakage and δ), but exceedance is 0.0% for every λ since the verifier enforces the budget regardless of reward; we select $\lambda = 1.0$ as the knee. Raising $\mu(\text{ADVERSARIAL}) \in \{3, 4, 6\}$ monotonically lowers realized leakage to adversarial sinks (0.71 $\rightarrow$ 0.49 $\rightarrow$ 0.33 bits) at higher fallback (Appendix D.2), confirming μ is a policy risk weight, not an information-theoretic quantity.

Overhead. The defender pass dominates per-release latency (Table 7: 142 ms at p_{50}); the verifier adds 62 ms at p_{50} and 249 ms at p_{99} , the credal audit its largest term. End-to-end mediation runs 204 ms (p_{50}) to 736 ms (p_{99}) — 18.3% wall-clock and 1.4 $\times$ token overhead over an unmediated trajectory, at 4.9 releases/s on one A100. Both verifier terms are bounded by construction: validate p_{99} stays under 100 ms up to rubric sets of size 12, and the bounded-treewidth audit keeps audit p_{99} under 150 ms (Appendix D.2).

TABLE 7. PER-RELEASE LATENCY (MS). EACH COLUMN DECOMPOSES THE REPRESENTATIVE TRACE AT THAT END-TO-END PERCENTILE, SO COMPONENT ROWS SUM TO THE END-TO-END ROW; THESE ARE NOT INDEPENDENT PER-COMPONENT PERCENTILES.

Component	p_{50}	p_{95}	p_{99}
Defender pass	142	318	487
Validate	23	61	98
Credal audit	37	89	143
Ledger append	2	5	8
End-to-end	204	473	736

6. Conclusion

We reframed privacy for LLM agents as *posterior-risk control*: rather than filtering individual outputs, OCELOT budgets how much an adversary’s belief about a protected secret may improve across a trajectory. Its mechanism, Witness-Verified Declassification, reconciles needing an LLM’s semantic judgment with being unable to trust it in an adversarial loop by demoting it to an untrusted witness producer under deterministic verification. This separates a budget guarantee that holds by ledger arithmetic regardless of model behavior from a semantic-privacy claim whose reliance on calibration we measure rather than assume, and empirically yields lower leakage at higher utility than prior defenses, with graceful degradation under adaptive attack. The lesson generalizes beyond privacy: where an agentic decision cannot safely rest on the model itself, an untrusted model proposing evidence that a sound, deterministic checker verifies—conservative fallback, not silent failure, as the default—is a reusable architecture.

Limitations. Every weakness below falls on the empirical side of that separation; none touches the deductive budget guarantee. The budget’s semantic reading is model-relative—as inference privacy inherently is—so we measure it directly: certified costs upper-bound leakage only as well as calibration transfers, and coverage falls against adversaries beyond our stress pool, contained though not eliminated by the calibration margin, an independent credal audit, and conservative policies on the few residual high-risk field-operator pairs (§5.4). Rubric coverage is validated, not proven complete, but this bounds utility, not safety: uncovered content is conservatively dropped, so incompleteness can only over-restrict, never leak. OCELOT-Traj-500 is synthetic by necessity—certified evaluation needs ground-truth secrets and necessity labels real corpora lack—but does not flatter our findings, which transfer to held-out rubrics, held-out categories, and independently annotated real traces (§5.3); production-scale evaluation is the principal open item. Finally, the deductive layer rests on a small trusted base—a deterministic verifier and harness, the LLM untrusted and defender weights trusted only at rest—the standard, here minimal, reference-monitor assumption; relaxing it and extending the operator algebra and field ontology to new domains are directions the framework is built to accommodate.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [2] J. Young, “Effective Harnesses for Long-Running Agents,” <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>, Nov. 2025, anthropic Engineering blog. Accessed: Jun. 1, 2026.
- [3] R. Lopopolo, “Harness Engineering: Leveraging Codex in an Agent-First World,” <https://openai.com/index/harness-engineering/>, Feb. 2026, openAI blog. Accessed: Jun. 1, 2026.
- [4] L. Pan, L. Zou, S. Guo, J. Ni, and H.-T. Zheng, “Natural-language agent harnesses,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.25723>
- [5] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, A. Oprea, and C. Raffel, “Extracting training data from large language models,” 2021. [Online]. Available: <https://arxiv.org/abs/2012.07805>
- [6] R. Staab, M. Vero, M. Balunović, and M. Vechev, “Beyond memorization: Violating privacy via inference with large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.07298>
- [7] F. E. Yagoubi, G. Badu-Marfo, and R. A. Mallah, “Agentleak: A full-stack benchmark for privacy leakage in multi-agent llm systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.11510>
- [8] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.13352>
- [9] N. Mireshtghallah, H. Kim, X. Zhou, Y. Tsvetkov, M. Sap, R. Shokri, and Y. Choi, “Can llms keep a secret? testing privacy implications of language models via contextual integrity theory,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.17884>
- [10] H. Nissenbaum, “Privacy as contextual integrity,” *Wash. L. Rev.*, vol. 79, p. 119, 2004.
- [11] Z. Ding, J. Li, Z. Lu, and J. Shi, “Flexguard: Continuous risk scoring for strictness-adaptive llm content moderation,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.23636>
- [12] S. Wang, F. Yu, X. Liu, X. Qin, J. Zhang, Q. Lin, D. Zhang, and S. Rajmohan, “Privacy in action: Towards realistic privacy mitigation and evaluation for llm-powered agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.17488>
- [13] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.18813>
- [14] P. Y. Zhong, S. Chen, R. Wang, M. McCall, B. L. Titzer, H. Miller, and P. B. Gibbons, “Rtbas: Defending llm agents against prompt injection and privacy leakage,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.08966>
- [15] A. Zharmagambetov, C. Guo, I. Evtimov, M. Pavlova, R. Salakhutdinov, and K. Chaudhuri, “AgentDAM: Privacy leakage evaluation for autonomous web agents,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. [Online]. Available: <https://openreview.net/forum?id=qaxf7q41aK>
- [16] S. Asif and M. M. Amiri, “Information-theoretic privacy control for sequential multi-agent llm systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.05520>
- [17] G. Smith, “On the foundations of quantitative information flow,” in *International Conference on Foundations of Software Science and Computational Structures*. Springer, 2009, pp. 288–302.
- [18] B. Köpf and D. Basin, “An information-theoretic model for adaptive side-channel attacks,” in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 286–296.
- [19] C. Dwork and A. Roth, “The algorithmic foundations of differential privacy,” *Foundations and trends® in theoretical computer science*, vol. 9, no. 3-4, pp. 211–487, 2014.
- [20] A. Darwiche, *Modeling and reasoning with Bayesian networks*. Cambridge university press, 2009.
- [21] D. D. Mauá, C. P. de Campos, A. Benavoli, and A. Antonucci, “Probabilistic inference in credal networks: new complexity results,” *Journal of Artificial Intelligence Research*, vol. 50, pp. 603–637, 2014.
- [22] C. Barrett, A. Stump, C. Tinelli *et al.*, “The smt-lib standard: Version 2.0,” in *Proceedings of the 8th international workshop on satisfiability modulo theories (Edinburgh, UK)*, vol. 13, 2010, p. 14.
- [23] L. De Moura and N. Bjørner, “Z3: An efficient smt solver,” in *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2008, pp. 337–340.
- [24] D. Melcer, N. Fulton, S. K. Gouda, and H. Qian, “Constrained decoding for fill-in-the-middle code language models via efficient left and right quotienting of context-sensitive grammars,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.17988>
- [25] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.03300>
- [26] C. Barrett and C. Tinelli, “Satisfiability modulo theories,” in *Handbook of model checking*. Springer, 2018, pp. 305–343.
- [27] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [28] Q. Team, “Qwen3.5: Accelerating productivity with native multimodal agents,” February 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [29] A. Chen, A. Li, B. Zhou, B. Gong, B. Jiang, B. Dan, C. Yu, C. Wang, C. Ma, C. Zhong *et al.*, “The minimax-m2 series: Mini activations unleashing max real-world intelligence,” *arXiv preprint arXiv:2605.26494*, 2026.
- [30] DeepSeek-AI, “Deepseek-v4: Towards highly efficient million-token context intelligence,” 2026.
- [31] G. Nelson and D. C. Oppen, “Simplification by cooperating decision procedures,” *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 1, no. 2, pp. 245–257, 1979.
- [32] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” *Journal of cryptographic engineering*, vol. 2, no. 2, pp. 77–89, 2012.
- [33] F. Pub, “Secure hash standard (shs),” *Fips pub*, vol. 180, no. 4, p. 2012, 2012.

Appendix A. Lemmas and Their Proofs

We prove the three body lemmas. All are model-relative: they bound quantities under Pre-Reg. A’s calibration model and the verifier’s construction, not true adversarial inference (Layer-2, §3.4).

Lemma 1 (model-relative sub-additivity). *Let $\sigma = (\sigma_1, \dots, \sigma_n)$ be the operator sequence authorized for candidate x , applied to pairwise-disjoint atoms, producing variant $v = \sigma(x)$, and let $\hat{L}(v; S)$ be its min-entropy advantage under the calibration model of Pre-Reg. A. Suppose each*

$C(o_i, f_i)$ is the level- $(1-\alpha)$ UCB on the incremental advantage of o_i given the cumulative view of $o_1, \dots, o_{i-1}$, where f_i is the secret field that o_i targets. Then

$$\widehat{L}(v; S) \leq \sum_{i=1}^n C(o_i, f_i).$$

Proof. By the Phase-1 typed-object schema the operators target pairwise-disjoint atoms, so v is determined by the views $y_i := o_i(a_i)$ and the shared residual context, which each calibration proxy of Pre-Reg. A observes as one object. For proxy j , write $M_i^{(j)} = \max_s \hat{P}_j(S=s \mid y_1, \dots, y_i)$ for its MAP posterior after the first i views and $M_0 = \max_s P(S=s)$ for the MAP prior. The per-step increments $\Delta_{i,j} = \log_2 M_i^{(j)} - \log_2 M_{i-1}^{(j)}$ telescope to that proxy's total advantage, $\widehat{L}_j(v; S) = \log_2 M_n^{(j)} - \log_2 M_0$, for any operator ordering. The calibration adversary is the strongest proxy, so

$$\begin{aligned} \widehat{L}(v; S) &= \max_j \widehat{L}_j(v; S) \\ &= \max_j \sum_i \Delta_{i,j} \leq \sum_i \max_j \Delta_{i,j} = \sum_i \Delta_i^*, \end{aligned}$$

using $\max$ of a sum $\leq$ sum of $\max$. Pre-Reg. A calibrates $C(o_i, f_i)$ as the $(1-\alpha)$ bootstrap UCB on the worst-proxy increment $\Delta_i^* = \max_j \Delta_{i,j}$ under the conditional distribution induced by the prior views, so $\Delta_i^* \leq C(o_i, f_i)$ with calibration-corpus coverage. Summing gives $\widehat{L}(v; S) \leq \sum_i C(o_i, f_i)$. $\square$

Remark. The per-increment UCBs hold at level $1-\alpha$ individually, so the length- n sequence bound holds at $\geq 1-n\alpha$ by a union bound; the UCBs are conservative in practice, and the operative guarantee is the release-level coverage measured in §5.4 (Table 4), with residual slack absorbed by the margin $\varepsilon(R)$ and the credal audit before any variant is accepted. Statistical dependence among atoms needs no separate assumption — Pre-Reg. A's incremental (and, for strongly dependent pairs, joint) calibration folds it into the cost.

Lemma 2 (polynomial credal-circuit bound under bounded treewidth). *Let G be the rubric's declared dependency graph over secret-field and atom variables with treewidth $\text{tw}(G) \leq w$, and let $\mathcal{C}$ be the credal sum-product circuit compiled from a tree decomposition of G , its credal sets specified by interval probabilities. The audit returns a sound upper bound on the credal upper-envelope max-posterior $\bar{P}_{\max}(S \mid v) = \max_s \sup_{P \in \mathcal{K}} P(S=s \mid v)$ in time $O(|\mathcal{C}|) = O(n \cdot k^{w+1})$, where $|\mathcal{C}|$ is the circuit size, n the number of variables, and k the per-variable domain bound.*

Proof. Precise sum-product inference on a width- w junction tree visits each circuit node once, costing the stated $O(|\mathcal{C}|)$ scalar operations [20]. For interval-specified credal sets, replacing each local factor by its interval-maximal value and propagating on the same schedule relaxes the joint credal set to the product of the local sets, so the propagated value upper-bounds the true upper envelope under strong-extension semantics [21] in $O(1)$ interval arithmetic per

node. Exact credal max-posterior is NP-hard in general [21], so the engine returns this conservative over-approximation — exactly what the Phase-3 reject rule needs, since rejecting when the bound exceeds $\text{cost} + \varepsilon(R)$ can only be conservative. $\square$

Remark. The audit engine compiles the rubric-declared dependency graph at rubric-publication time; rubrics whose declared G exceeds the treewidth budget w fail rubric-validation (Pre-Reg. C). Production uses $w \leq 7$ and $k \leq 16$, and the worst-case $O(n \cdot k^{w+1})$ bound is far above the sparse-graph audit p_{99} (Table 7).

Lemma 3 (decidability of the rubric DSL via QF_UFLRA). *Every rubric clause produced by Pre-Reg. C's authoring process compiles to a quantifier-free formula in linear real arithmetic with uninterpreted functions (QF_UFLRA). Satisfiability over the verifier's bounded atom domain A' is decidable, with worst-case cost at most exponential in formula size and empirically polynomial-time on the rubrics in our library.*

Proof. The DSL's five constructs each compile to QF_UFLRA. *Trigger predicates* (regex, NER-type, lexicon membership) are evaluated on the candidate and substituted as ground booleans. *Necessity, operator-coverage*, and *utility* clauses have the form $\forall_{a \in A'} \phi(a) \Rightarrow \psi(a)$ over the uninterpreted functions field, label, operator, necessity, unrolled over the finite $A' = \{a_1, \dots, a_m\}$ to $\bigwedge_j (\phi(a_j) \Rightarrow \psi(a_j))$. *Budget arithmetic* adds linear real inequalities over per-operator cost variables with rubric-supplied constants. The result is a Boolean combination of atomic LRA (in)equalities over uninterpreted-function applications — the QF_UFLRA fragment, decidable by a SAT solver, a linear-arithmetic procedure, and Nelson–Oppen [22], [31]; formula size is linear in $|A'|$ and the per-rubric clause count, both bounded by Pre-Reg. C. $\square$

Appendix B. Theorems and Their Proofs

Theorem SP-1 (Layer-1a deductive non-exceedance). *For every trajectory authorized by OCELOT, the cumulative ledger-recorded cost satisfies $\sum_{i=1}^k \text{cost}_i \leq B$, where $B = B_0$ is the fixed per-trajectory base budget of the matched rubric set; the trust multiplier $\mu(z)$ enters only through cost_i (Eq. 8), not B . This holds without any assumption on the defender's behavior, calibration soundness, or audit-engine correctness.*

Proof. The Commit phase (§4.4) authorizes a release at step k only if $L_{\text{cum}}^{(k-1)} + \text{cost}_k \leq B$, with $L_{\text{cum}}^{(k-1)} = \sum_{i=1}^{k-1} \text{cost}_i$ and B fixed. By induction with base $L_{\text{cum}}^{(0)} = 0$: assume $\sum_{i=1}^{k-1} \text{cost}_i \leq B$. At step k , either the verifier authorizes, giving $\sum_{i=1}^k \text{cost}_i \leq B$, or it rejects, appending no entry and leaving the cumulative cost unchanged. The prior costs are read from the Merkle-chained, signed ledger (tampering caught by Phase 1 or Theorem SP-2); cost_k is computed fresh by the deterministic Cost phase from the calibrated table $C(\text{op}, f)$, the structural-field-augmented set A' , the max-

over-fields rule, and the rubric’s $\mu(z)$ — all from trusted rubric and metadata, none depending on the defender. $\square$

Theorem SP-2 (Ledger tamper-evidence). *Under EUF-CMA security of Ed25519 and collision resistance of SHA-256, any modification, reordering, deletion, or insertion in the ledger $\mathcal{L}$ of an authorized trajectory is detected with overwhelming probability by external audit.*

Proof sketch. Each entry e_i embeds $\text{prev_entry_hash}_i = \text{hash}(e_{i-1})$ and is signed under sk_V over its full content including the chain link. Any content change is caught by signature verification; any insertion, deletion, or reordering breaks a successor’s prev_entry_hash and is caught by chain re-traversal. Evasion requires forging an Ed25519 signature or a SHA-256 collision, both negligible under standard assumptions [32], [33]. $\square$

On Layer 2. We prove no theorem about Layer 2. The implication $H_\infty(S) - H_\infty(S \mid \text{view}) \leq B$ holds only when the calibrated $C(\text{op}, f)$ uniformly upper-bounds true adversarial min-entropy leakage on released variants — an empirical claim. The evaluation reports the residual gap between the calibration model and Pre-Reg. B’s adversary classes; a deductive proof would require bounding the true posterior under unbounded compute and arbitrary background knowledge, which no defense can soundly establish; following quantitative information-flow practice [18], we report it rather than overclaim.

Appendix C. Pre-Registrations

Each protocol is content-hashed and posted publicly before any data collection or model training; deviations are disclosed with timestamp and justification.

C.1. Pre-Reg. A: Cost calibration protocol

Object. For each rubric r and admissible (operator, secret-field) pair (op, f) , calibrate $C(\text{op}, f)$ as a $1-\alpha$ UCB ($\alpha = 0.05$) on the per-instance *incremental* advantage op contributes to a field- f atom (Estimation below).

Corpus. For each (op, f) , sample $N \geq 500$ candidates with known field-to-span ground truth under the trajectory-sampling protocol of §4.2, partitioned at sampling time into calibration (80%) and a sealed held-out set (20%) used only for coverage reporting.

Proxy-adversary pool. The calibration pool is a panel of open-weight models spanning distinct scales (names and versions fixed at registration), each prompted as an inference adversary that outputs a discrete posterior over the rubric-declared secret-field population.

Estimation. For operator o_i at position i in a sampled sequence, proxy j observes the cumulative view and reports the incremental advantage $\Delta_{i,j} = \log_2 M_i^{(j)} - \log_2 M_{i-1}^{(j)}$ of Lemma 1’s proof (empty-prefix baseline $M_0 = \max_s P(S=s)$ for $i=1$). The instance value is the worst proxy $\Delta_i^* = \max_j \Delta_{i,j}$; $C(o_i, f_i)$ is the bootstrap percentile UCB (5,000 resamples) at level $1-\alpha$ on $\{\Delta_i^*\}$ across field- f_i atoms.

Compositional calibration. Sequential dependence is captured by the incremental conditioning above. For the few strongly dependent atom pairs declared per rubric at registration, we calibrate a single joint cost $C(o_1, f_1, o_2, f_2)$ applied in place of the pair’s two incremental terms — a slack-reducing tightening, not a soundness requirement.

Stress test. A separate *calibration-stress* pool, declared at registration but used only after calibration freezes, includes at least one model strictly more capable than any in the calibration pool. Calibration is valid only if the stress adversary’s measured advantage exceeds $C(\text{op}, f)$ by more than $\Delta = 0.5$ bits on at most 5% of held-out instances; failure triggers re-calibration.

Trust-class multipliers. Deployment values $\mu(\text{REQUIRED-SERVICE})$, $\mu(\text{SEMI-TRUSTED})$, and $\mu(\text{ADVERSARIAL})$ are fixed at 1, 2, 4 (all ≥ 1).

C.2. Pre-Reg. B: Residual escape rate δ measurement

Object. The empirical probability δ that the defender’s labels pass the validate phase while misclassifying an atom’s secret field outside the verifier-considered set $F_V \cup F_D$ — the residual that can break the Layer-1b cost-assignment bound. (The distinct event “true leakage exceeds certified cost,” bearing on Layer-2 calibration, is measured separately in §5.4.)

Adversary classes. Three omission/relabel vectors are pre-registered, each making the defender assign a secret field outside $F_V \cup F_D$ to an atom that still passes validate. $\mathcal{A}_{\text{obfuscate}}$ embeds the sensitive span in obscure phrasing so the defender emits a NONE or wrong-field label (pool: published AgentLeak [7] patterns plus per-rubric templates); $\mathcal{A}_{\text{distribute}}$ spreads a secret across spans so no single span triggers a coverage predicate (per-rubric distribution templates); $\mathcal{A}_{\text{relabel}}$ perturbs task context to induce a wrong field or permissive necessity (e.g., DOB framed as flight-search-relevant; necessity-inversion templates). Injection, compositional joint-disclosure (“DictAttack-class” [7]), and multi-sink collusion instead target the wrapper, cost bound, and budget, and are evaluated in §5.5 and §5.4.

Sample size. For each (adversary class, rubric), draw $K = 1,000$ crafted attacks and run OCELOT, recording per-trajectory escape per the Object above.

Reporting. Per (rubric, adversary class), report the escape rate and its Clopper-Pearson 95% interval; the aggregate δ for the library is the maximum across (rubric, adversary class) pairs.

Pre-committed threshold. The defense passes Layer-1b validation iff aggregate $\delta \leq 0.05$ on every pre-registered adversary class. Failure triggers diagnostic reporting with no Layer-1b claim; no post-hoc adversary-class exclusion.

C.3. Pre-Reg. C: Rubric library construction protocol

Object. A versioned rubric library $\mathcal{R}$ covering the evaluation’s task categories, each rubric satisfying the seven-component schema of §4.1.

Stage 1 — Drafting. Each rubric is authored by a domain-expert annotator following a written template. Frontier-model assistance is permitted as a research tool (suggesting trigger predicates, listing field-population samples), but no content enters the rubric without expert review.

Stage 2 — Validation. Each draft is checked against ≥ 100 realistic agent trajectories per task category (from a corpus disjoint from the training and δ -measurement corpora) on three pre-committed thresholds: coverage-obligation hit rate ≥ 0.95 , necessity Krippendorff $\alpha \geq 0.80$ across three independent annotators, and operator-algebra completeness (no rubric atom without an admissible operator) = 1.0. Any failure returns the rubric to drafting.

Stage 3 — Adversarial probing. A red team (≥ 20 hours per rubric) attempts releases that satisfy all coverage obligations yet leak above calibrated cost; successful attacks patch the rubric (typically new coverage obligations or adjusted necessity clauses) and return it to Stage 2.

Challenge-pool construction. Per rubric, a pool of ≥ 50 challenge predicates (for two-pass consistency checking, §4.2) is built by the author and red team, content-hashed and sealed before defender training; runtime draws are uniform.

Annotator profile. Annotators are security/privacy graduate or postdoc researchers trained on the rubric template, schema, and field ontology; a 50-pair pilot precedes Stage-2, with sub-threshold dimensions triggering documented deviation and rule clarification (App. D.1).

Signing and versioning. Each finalized rubric is content-hashed (SHA-256), signed under the rubric authority’s private key, and published to a versioned registry. Modifications produce a new version with a new content hash; the verifier accepts only registered, signature-verified rubrics by hash.

C.4. The Defender’s Label-and-Proposal Object

The object (Figure 8) separates a trusted, V -verified *release context* — action id, channel type, destination and trust class, the signed rubric, and the `prev_hash` chaining it into the ledger, all from interceptor metadata and the signed registry — from the *defender evidence* V re-checks, each atom carrying a `provenance` flag beside its (field, kind, necessity, operator) label. It is the interface through which the *verifier*, not the defender, neutralizes the Figure 1 threats. *Injection:* `channel_type` selects the mediation path, inbound output being wrapped in an authority-stripping block regardless of its label and a forwarded `USER-LOCAL` atom mediated as an act release. *Accumulation:* `prev_hash` binds the object to the Merkle ledger, and the incremental cost charges each release on its advantage given the cumulative released view (Lemma 1) before Commit enforces $L_{\text{cum}} + \text{cost} \leq B$, so splitting a secret across steps does not evade its cost. *Collusion:* that ledger and budget span all channels — memory writes route to $\mathcal{L}_{\text{mem}}$ at the most adversarial μ and draw on the same B — and each release is costed against the union of prior cross-sink releases, so sinks pooling receipts still divide one fixed B . A jailbroken labeling is caught by the two-pass challenges,

V ’s re-firing of the rubric’s coverage triggers, and its max-over-fields re-check (residual δ).

Appendix D. Experimental Details

D.1. Realized Protocol Values (Pre-Reg. A–C)

Dataset and annotation (C). OCELOT-Traj-500 spans 12 task categories (travel booking, medical-record retrieval, financial onboarding, etc.) over an eight-type secret-field ontology (LEGAL-NAME, DOB, PAYMENT-TOKEN, DIAGNOSIS, ADDRESS, PHONE, EMAIL, ACCOUNT-ID), generated by the synthetic pipeline of §4.2 with per-category sink-trust proportions over {ADVERSARIAL, SEMI-TRUSTED, REQUIRED-SERVICE} fixed in Pre-Reg.C. In the 50-pair pilot, two blinded annotators labeled per-atom field and necessity at per-category Krippendorff α of 0.74–0.89 (median 0.82; necessity hardest at 0.74, field easiest at 0.86); the necessity 0.74 fell below the 0.80 gate, was logged as a deviation, and prompted a necessity-rule clarification in the three affected categories before three-annotator Stage-2 met the gate. **Calibration (A).** The per-rubric margin $\varepsilon(R)$ has mean 0.18 (range 0.11–0.27 bits). The calibration pool is {Llama-3.3-70B, Qwen-3.5-27B}; the held-out stress pool {MiniMax-M2.5, DeepSeek-V4-Pro}, frozen before measurement, is strictly stronger — top-1 0.84/0.86 versus 0.71/0.73 on a secret-inference probe, and GPQA-Diamond 71.2/74.6 versus 50.5/53.8. **Escape rate (B).** δ is measured with 4-of-50 two-pass challenges per release over five seeds, inter-pass disagreement routed to fallback.

D.2. Baselines, Sensitivity, and Scalability

Baselines. AGENTDAM* augments the monitoring estimator with a block-if-leakage-exceeds- τ policy, τ tuned per benchmark to match OCELOT’s fallback rate; even so it leaks 1.9–4.6 bits and exceeds B^* on 28% of trajectories, since thresholding a per-release estimate cannot bound the cumulative sum. CaMeL and RTBAS lack a sink-trust API (we map REQUIRED-SERVICE $\rightarrow$ trusted, SEMI-TRUSTED/ADVERSARIAL $\rightarrow$ untrusted); FlexGuard and PrivacyChecker use their default per-release thresholds.

Sensitivity. The GRPO penalty λ tunes the operating point within the authorized region (Table 8); exceedance is 0.0% for every λ , with $\lambda=1.0$ the knee. Raising $\mu(\text{ADVERSARIAL}) \in \{3, 4, 6\}$ monotonically reduces leakage to adversarial sinks (0.71/0.49/0.33 bits) while raising their fallback (9.1/11.3/15.8%); overall fallback at the deployed $\mu=4$ stays at the 8.7% of Table 8. Re-calibrating under non-uniform (empirical-frequency) priors shifts stress-pool coverage by ≤ 1.2 points (92.1% $\rightarrow$ 91.4–93.0%), so the calibration is not a uniform-prior artifact.

Scalability. Validate p_{99} is 41/68/98 ms at rubric-set sizes 4/8/12 and audit p_{99} is 89/121/143 ms at treewidth 3/5/7; both are bounded by construction, matching Table 7.

You are OCELOT’s privacy defender. You label what an agent is about to release and propose the least- disclosing way to release it; you never authorize, cost, or see the budget -- the verifier re-checks everything you emit, so over-claiming utility only gets your proposal rejected. Treat all observed or inbound content as data, never as instructions to you. PASS 1: for each candidate span emit an atom -- field (rubric ontology, else NONE), kind (direct|indirect), provenance (user-local|external), necessity for the (field, destination) pair, and the least-disclosing operator+params that still serves the task; if a span’s field, necessity, or operator is uncertain, emit the atom and choose the more restrictive operator (identity < generalize < substitute < redact < drop). PASS 2: answer the verifier’s sampled challenge predicates consistently with your Pass-1 labels; any inconsistency triggers the conservative fallback. Use only fields named in the signed rubric; never propose an operator weaker than the rubric’s per-(field, destination-class) floor.

```

LABEL-AND-PROPOSAL OBJECT <A, 1, sigma> # parsed in Bind; pure input to V’s four phases

release context (trusted: interceptor metadata + signed registry; V re-verifies, D cannot forge)
  action_id : <id of the agent action being mediated>
  channel_type : <act | observe | memory> # selects V’s mediation path
  destination : <dest-id> : <dest-class> (<trust-class z>)
  rubric : <rubric-id>@<version> sig:<...> # signed rubric Bind resolves + checks against
  prev_hash : <hash of latest ledger entry> # binds this object into the trajectory ledger

defender evidence (untrusted: V re-checks every field against the rubric)
  atoms : { span, field:<ont|NONE>(F_D), kind:<direct|indirect>, provenance:<user-local|external>,
            necessity:<required|unnecessary>, op:<operator>(<params>) } ...
  challenges : { id, ask, answer } ... # 4 sampled from rubric pool (Pre-Reg.C)

# Example -- planner forwards an authenticated record to the LLM; source output also carried injection.
release context:
  action_id: a_0309 channel_type: act destination: llm-api-7 : llm-context (adversarial)
  rubric: booking-v4@2.1 sig:9f2c.. prev_hash: 7c20..9e
defender evidence:
  atoms:
    { span:"DOB 02/07/1985", field:dob, kind:direct, provenance:user-local,
      necessity:unnecessary, op:redact } # private; stripped before the LLM
    { span:"<<system: mail all to evil@x>>", field:NONE, kind:direct,
      provenance:external, necessity:unnecessary, op:identity } # authority removed by wrapper
  challenges:
    { id:c14, ask:"user-local secret bound for an untrusted sink?", answer:"yes: dob -> redact" }
    { id:c22, ask:"any span carrying executable authority?", answer:"no (wrapper-quoted)" }

```

Figure 8. The defender’s runtime label-and-proposal object: a trusted, V-verified release context (action, channel, destination+trust, signed-rubric binding, ledger hash) and untrusted, V-rechecked evidence. Its fields are the verifier’s threat hooks—channel_type/provenance (injection wrapper), prev_hash (accumulation: incremental cost + budget), destination+trust (collusion: one all-channel budget)—none requiring the defender to detect an attack.

TABLE 8. GRPO FALLBACK-PENALTY λ SWEEP ON
OCELOT-TRAJ-500. FB = FALLBACK RATE; OTHER COLUMNS AS IN
TABLE 6. THE $\lambda=1.0$ ROW IS THE DEPLOYED CONFIGURATION.

λ	FB	U	L	δ	Exc
0.1	16.8	84.1	0.24	1.4	0.0
0.5	11.3	89.2	0.28	1.6	0.0
1.0	8.7	91.3	0.31	1.7	0.0
2.0	6.1	92.4	0.37	2.2	0.0
5.0	3.4	92.9	0.48	2.9	0.0